

Table of Contents

1. Overview
2. Architecture
3. DocTypes
4. Roles & Permissions
5. Workflows
6. Business Flows
7. Validations

Visitor Management System

Complete Technical Documentation

Frappe / ERPNext Application
Generated 2026-05-07

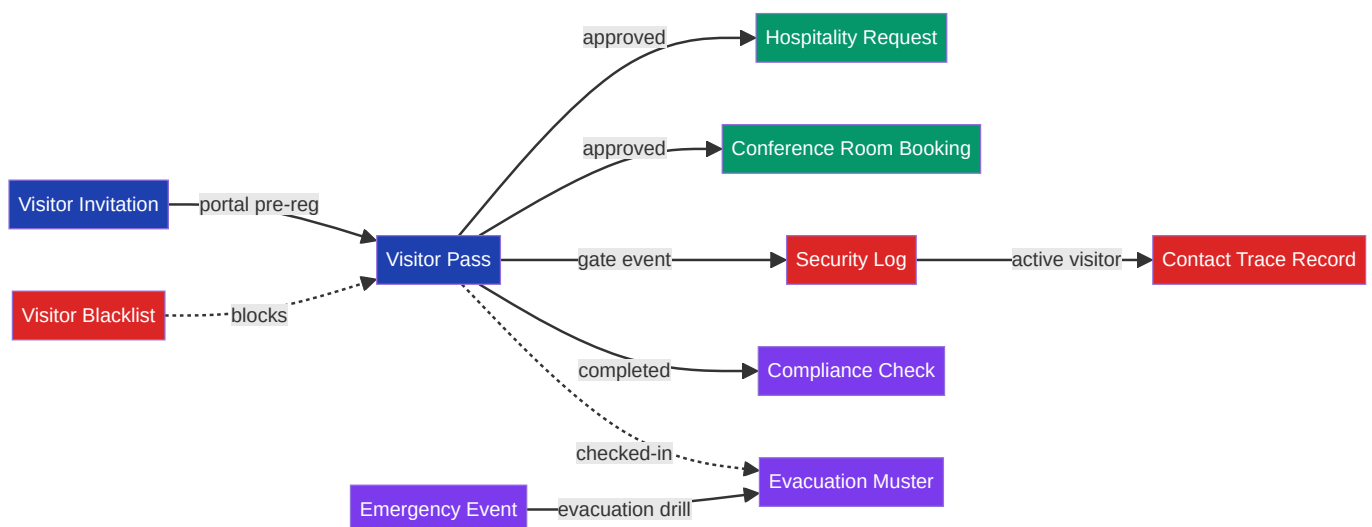
Overview

A Frappe-based application for managing the complete lifecycle of corporate visitors: pre-registration, multi-lane approval, gate check-in/out, hospitality arrangements (meals, cabs, hotels, factory tours), conference room bookings, security incidents, emergency evacuation, and compliance audit trails.

Table of Contents

#	Document	What's inside
1	architecture.md	System overview, module structure, doctype relationships, data flow diagrams
2	doctypes.md	Every doctype, every field, what each one represents
3	roles-and-permissions.md	The 11 roles, who can do what, the per-doctype permission matrix
4	workflows.md	The 3 approval workflows with full state-machine diagrams
5	business-flows.md	End-to-end sequence diagrams: visit lifecycle, hospitality, room booking, emergency
6	validations.md	Every business rule the system enforces, with examples

Quick Concept Map



Three big things to understand:

1. **Visitor Pass** is the central record. Everything orbits around it.
2. **Three workflows** govern approvals — Visitor Pass, Hospitality Request, Conference Room Booking — each routes to a different role lane based on the visitor type or booking type.
3. The **"VP-must-be-Approved" gate** is the real-world rule: a hospitality request or room booking linked to a visitor cannot move to "Pending Approval" until the underlying Visitor Pass is itself Approved. No catering arrives for a visitor that isn't confirmed.

Quick Start

For an end user

- **Reception / Employee** wanting to host a visitor → see [business-flows.md](#) → [Visitor Lifecycle](#).
- **Approver** (Sales/HR/System Manager/HOD/CEO) → open the **Visitor Pass** list, filter by `Pending <YourRole>`, click into each pass and use the **Approve / Reject** workflow buttons.
- **Hospitality Manager / Facility Manager** → see [business-flows.md](#) → [Hospitality Flow](#).
- **Security** → use the **Security Log** to record check-in / check-out events at the gate. See [business-flows.md](#) → [Gate Operations](#).

For an administrator

- **First-time setup** → [architecture.md](#) → [Setup Checklist](#) and configure [VMS Settings](#).
- **Add a Conference Room** → see [doctypes.md](#) → [Conference Room](#).
- **Manage permissions / add a user** → [roles-and-permissions.md](#) → [Adding a User](#).

For a developer

- **Project layout** → see [architecture.md](#) → [Repo Layout](#).
- **Run tests** → `bench --site <site> run-tests --module visitormanagement.tests.test_regression`. See [validations.md](#) → [Regression suite](#).
- **Extend the workflow** → see [workflows.md](#) → [Modifying a Workflow](#).

App Facts

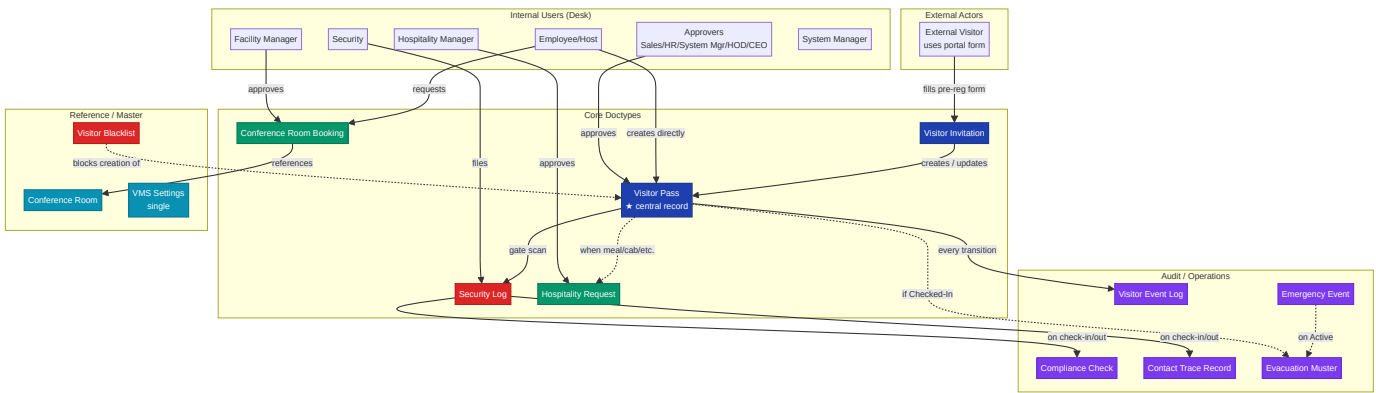
Fact	Value
Frappe app name	<code>visitormanagement</code>
Modules	Visitor Management , Conference Room
Total doctypes	16 (12 documents, 1 single, 3 child tables)
Workflows	3 (Visitor Pass / Hospitality Request / Conference Room Booking)
App-defined roles	11
Public portal forms	1 (Visitor Pre-Registration)
Test suite size	45 regression tests

Conventions used in this documentation

-  = the role / actor responsible for an action
-  = explicitly denied / blocked
-  = real-world business rule worth highlighting
- All field names are in `inline_code` matching the doctype JSON
- All Mermaid diagrams render directly on GitHub, in VS Code with a Mermaid plugin, or anywhere markdown supports it.

Architecture

High-level system map

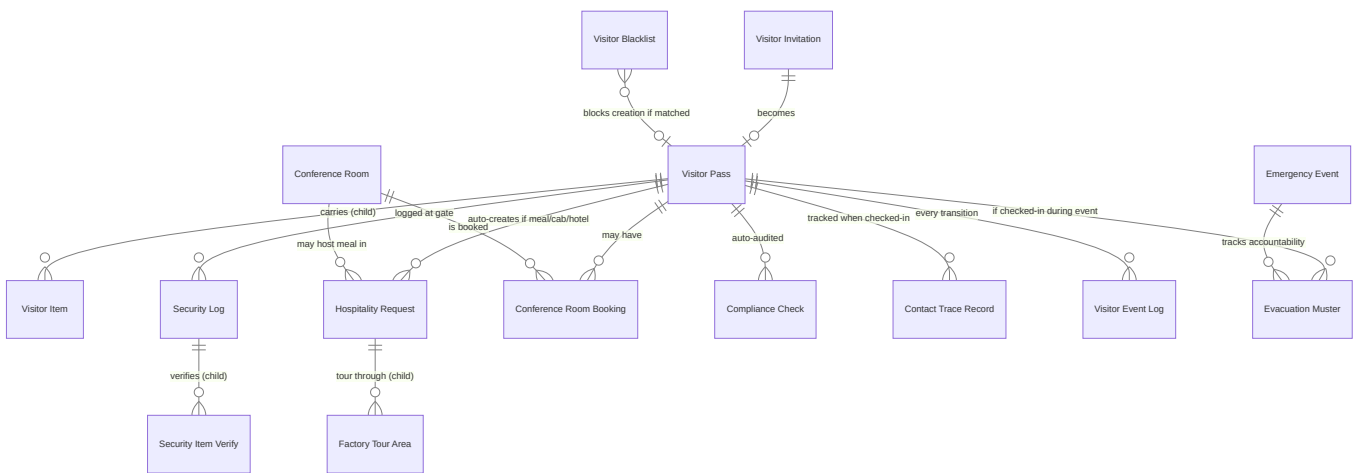


Modules

The app ships **two Frappe modules** (top-level groupings):

Module	Doctypes
Visitor Management	Visitor Pass, Visitor Invitation, Security Log, Hospitality Request, Visitor Blacklist, Visitor Event Log, Compliance Check, Contact Trace Record, Emergency Event, Evacuation Master, VMS Settings (single), Visitor Item (child), Security Item Verify (child), Factory Tour Area (child)
Conference Room	Conference Room, Conference Room Booking

Doctype relationship diagram



Legend: `||--o{` = one-to-many, `||--o|` = one-to-one (optional), `}o--o|` = blocking reference.

Data flow — what happens when



Repo layout

```
visitormanagement/
├─ visitormanagement/      # the app package
│   ├─ modules.txt         # ["Visitor Management", "Conference Room"]
│   ├─ hooks.py            # event handlers, scheduled jobs, fixtures
│   ├─ permissions.py      # ★ has_visitor_pass_permission Python hook
│   ├─ patches.txt         # post-install/migrate patches
│   └─ visitor_management/
│       └─ doctype/        # 14 doctypes (12 doc + 1 single + 3 child)
│           └─ visitor_pass/ # central doctype + .py controller + .js form
│               └─ visitor_invitation/
│                   └─ security_log/
│                       └─ hospitality_request/
│                           └─ visitor_blacklist/
│                               └─ visitor_event_log/
│                                   └─ compliance_check/
│                                       └─ contact_trace_record/
│                                           └─ emergency_event/
│                                               └─ evacuation_muster/
│                                                   └─ vms_settings/ # single doctype
│                                                       └─ visitor_item/ # child table
│                                                           └─ security_item_verify/ # child table
│                                                               └─ factory_tour_area/ # child table
│   └─ lifecycle.py        # ★ shared business logic + side-effect hooks
│   └─ portal.py           # public pre-registration endpoint
│   └─ candidate_flow.py   # Job-Applicant → Visitor Invitation auto-flow
│   └─ tasks.py            # daily hospitality digest cron
│   └─ validators.py       # ID-format checkers (Aadhaar/PAN/Passport)
│   └─ web_form/visitor_pre_registration_form/ # public portal form
│   └─ workspace/visitor_management/ # desk sidebar
│   └─ visitor_management_dashboard/ # security dashboard
│   └─ dashboard_chart/ # 4 charts
│   └─ number_card/ # 6 KPI cards
├─ conference_room/
│   └─ doctype/
│       └─ conference_room/
│           └─ conference_room_booking/
├─ fixtures/
│   └─ role.json # 11 app roles
│   └─ workflow.json # 3 workflows
│   └─ workflow_state.json
│   └─ workflow_action.json
├─ patches/post_model_sync/ # one-off migration patches
├─ tests/
│   └─ test_regression.py # 45 tests covering the contract
└─ docs/ # this documentation
```

Hooks wired in `hooks.py`

Hook	What it does
<code>permission_query_conditions["Visitor Pass"]</code>	SQL filter — each role only sees passes in its lane
<code>has_permission["Visitor Pass"]</code>	Per-doc Python check — Security read-only, lane-based access
<code>doc_events["Job Applicant"]["after_insert"]</code>	Auto-create Visitor Invitation for offline interviews
<code>scheduler_events["cron"]["0 7 * * *"]</code>	Daily 7 AM hospitality digest email
<code>webform_include_js["visitor-pre-registration-form"]</code>	ID validators on the portal
<code>fixtures</code>	Roles + workflows + workflow states/actions auto-loaded
<code>after_migrate</code>	Re-runs notification config sync

Setup checklist

Once the app is installed on a fresh site (`bench install-app visitormanagement`):

1. **Bench migrate** — applies patches: workflows, roles, custom fields, workspace, dashboard
2. **Open VMS Settings** at `/app/vms-settings` and fill: - `admin_email` (operations contact) - `food_dept_email` (catering contact) - `default_checkout_time` - `gate_list` (one gate per line) - Toggle safety flags (`require_visitor_photo`, `qr_scan_required_at_gate`, etc.)
3. **Create Conference Rooms** at `/app/conference-room` — at least one Active room is needed before bookings can be made. Set `available_from/available_to` and `min/max_booking_*`.
4. **Create users** for each role you need: - Sales Manager (approves Customer passes) - HR Manager (approves Candidate passes) - System Manager (approves Contractor + Supplier passes; full admin) - HOD + CEO (approve VIP passes — two-step) - Security (gate operations) - Hospitality Manager (approves Hospitality Requests) - Facility Manager (approves Conference Room Bookings) - Employees (regular hosts)
5. **Provision Employee records** for each user — without one, Frappe auto-strips the Employee role. The patch `ensure_visitor_management_roles.py` does this automatically for Security/Hospitality/Facility users.
6. **Configure outgoing Email Account** at `/app/email-account` — needed for visitor invitations and approval notifications. (App code wraps every send in try/except so missing email config does NOT break workflows; only emails are lost.)

Background side-effects (not user-visible)

These run automatically on relevant events:

Trigger	Side effect
Visitor Pass <code>validate()</code>	<code>normalize_visitor_pass()</code> aligns workflow lane, builds badge number, syncs items, computes meal plan
Visitor Pass <code>on_submit()</code>	Generates QR code + sends approval email + creates Hospitality Request if meal flagged
Visitor Pass approval with <code>meal_required=1</code>	<code>ensure_hospitality_request()</code> auto-creates the Hospitality Request linked to the pass
Security Log Check-In	<code>sync_contact_trace()</code> + <code>sync_compliance_check()</code> + Visitor Event Log entry
Security Log Check-Out	Updates pass status + Visitor Event Log entry
Emergency Event → Active	<code>generate_emergency_muster_records()</code> creates Evacuation Muster row per checked-in visitor
Daily 7 AM cron	<code>send_daily_hospitality_digest()</code> emails Hospitality + Facility staff their day's tasks

DocTypes

Detailed reference for every doctype the app ships. For each, you get: - **Purpose** in one sentence - **Type** (document / single / child table) - **Key fields** with their meaning - **Links** to other doctypes - **Side-effects** that fire on save

For the Frappe-native field metadata (fieldtype, options, depends_on, etc.) consult the JSON at `visitorsmanagement/<module>/doctype/<name>/<name>.json`.

Quick Index

#	Doctype	Type	Purpose
1	Visitor Pass	doc	Central record for any visitor entering the facility
2	Visitor Invitation	doc	Pre-visit invitation sent to visitor with portal link
3	Security Log	doc	Gate event tracking — check-in, check-out, transfers
4	Hospitality Request	doc	Consolidates meal/cab/hotel/tour/buggy/greeting arrangements
5	Conference Room	doc	Master list of meeting rooms
6	Conference Room Booking	doc	Time-slot reservation against a Conference Room
7	Visitor Blacklist	doc	Block list for previously banned visitors
8	Visitor Event Log	doc	Audit trail entry per workflow/state event
9	Compliance Check	doc	Auto-built per-visitor compliance scorecard
10	Contact Trace Record	doc	Tracks where a visitor moved during their visit
11	Emergency Event	doc	An evacuation/safety event
12	Evacuation Muster	doc	One row per checked-in visitor during an Emergency Event
13	VMS Settings	single	App-wide configuration
14	Visitor Item	child	Items declared by visitor + verified at gate
15	Security Item Verify	child	Per-item verification record on a Security Log
16	Factory Tour Area	child	Areas covered in a factory tour

Visitor Pass

Purpose: The central record of every visitor who comes (or might come) to the facility. **Type:** Document, submittable (`is_submittable: 1`), naming `VP-.YYYY.-.#####` **Workflow:** [Visitor Pass Approval](#) (8 states, lane-based by `visitor_type`)

Field groups

Header / classification

Field	Type	Meaning
visitor_type	Select	One of: Customer, Candidate, Contractor, Supplier, VIP. Drives which approver lane the workflow routes to.
entry_type	Select	New / Existing — for repeat visitors
existing_visitor_pass	Link → Visitor Pass	When entry_type = Existing , link to the prior pass; details are pulled forward
request_channel	Select	Desk / Portal / Email / Phone / Walk-In
risk_level	Select	Low / Medium / High — auto-derived from visitor_type by infer_risk_level()
status	Select	Draft / Pending Approval / Approved / Items Verified / Checked-In / Checked-Out / Cancelled
workflow_state	Link → Workflow State	Detailed lane state (e.g. "Pending Sales Manager")

Identity

Field	Type	Meaning
visitor_full_name	Data, required	Visitor's full name
mobile_number	Phone, required	Used for OTP / contact / visitor_summary
email_id	Data, required	Validated against email regex
id_proof_type	Select, required	Aadhaar / PAN Card / Passport / Driving License
id_proof_number	Data, required	Format-validated per type (Verhoeff for Aadhaar, regex for PAN, etc.)
id_proof_scan	Attach Image, required	Scan/photo of the ID document
visitor_photo	Attach Image, required	Recent passport-size photo

Visit

Field	Type	Meaning
person_to_visit	Link → Employee, required	The host. Validated to be Active.
host_department	Data, fetched	Auto-filled from Employee
visit_date	Date, required	Must be today or up to 90 days in future
expected_checkin	Time, required	Must be < expected_checkout
expected_checkout	Time, required	
purpose_of_visit	Small Text, required	Free text
vehicle_number	Data	Optional — for parking/security
multi_day_pass	Check	If 1, pass is valid for several days; pass_valid_until becomes mandatory
pass_valid_until	Date	End date of the multi-day pass

Visitor type details (one of these tabs becomes visible based on **visitor_type**)

Customer-only fields

Field	Type	Meaning
company__organisation	Data	Visiting company
crm_reference_type	Select	Lead / Opportunity / Customer (or empty)
crm_lead_opportunity	Dynamic Link	Linked CRM record
visit_category	Select	Product Demo / Negotiation / Site Visit / Complaint / Audit
sales_executive	Link → Employee	Sales rep handling the meeting
products_discussed	Small Text	What was talked about
meeting_outcome	Select	Pending / Deal Closed / Follow-Up Needed / No Interest

Candidate-only fields

Field	Type	Meaning
job_applicant_link	Link → Job Applicant	Auto-linked when invitation came from HRMS Job Applicant flow
position_applied	Data	Role being interviewed for
candidate_interview_type	Select	Walk-In / Scheduled / Technical Round / Group Discussion
interview_panel	Small Text	Names of panel members

Contractor-only fields

Field	Type	Meaning
contractor_link	Link → Supplier	If contractor is a registered Supplier
work_order_ref	Link → Work Order	Reference to the work being done
tools_list	Small Text	What tools they're bringing

Supplier-only fields

Field	Type	Meaning
supplier_link	Link → Supplier	The Supplier master
supplier_visit_mode	Select	Meeting / Audit / Service
meeting_subject	Data	If mode = Meeting

VIP-only fields

Field	Type	Meaning
vip_category	Select	Government Official / Investor / Board Member / Media / Partner / Guest Speaker
mdceo_notified	Check	Required = 1 before CEO can approve. Real-world rule.
interpreter_required	Check	
interpreter_language	Select	English / Hindi / Tamil / Japanese / German / etc.
protocol_notes	Text Editor	VIP escort / lounge / parking instructions

Hospitality flags

These trigger auto-creation of a [Hospitality Request](#) on approval.

Field	Meaning
meal_required	Visitor needs lunch / breakfast / dinner
meal_type	Breakfast / Lunch / Dinner / All Day
cab_required	Pickup / Drop / Both
hotel_required	Hotel booking needed
factory_tour_required	Plant tour scheduled
buggy_required	Buggy needed (auto-set if factory tour)
greeting_required	Greeting/gift on arrival (auto-set for VIP)
conference_room	Link → Conference Room — booking room for the meeting
nda_required	NDA signing needed

Approval / gate (auto-populated, mostly read-only)

Field	Meaning
approval_date	Datetime when CEO/Approver clicked Approve
approved_by	User who approved
badge_number	Auto-generated on approval (e.g. CUS-2026-00042)
badge_colour	Visual code on the printed badge (Green/Orange/Purple/Teal/Gold)
qr_code_image	PNG embedded in the approval email
gate_verified_photo	Photo captured at the gate during check-in
gate_verified_on	Timestamp
gate_verified_by	Security officer who verified
actual_checkin / actual_checkout	Datetimes from Security Log events
current_location	Last gate visited

Items

Field	Meaning
items_carried	Free-text summary (auto-derived from visitor_items rows for portal submissions)
visitor_items	Child Table → Visitor Item — structured rows the visitor declared (laptop, USB, etc.)

Side-effects that fire on save / submit

- **validate()** runs: `normalize_visitor_pass()`, `_align_workflow_lane_with_visitor_type()`, `_validate_schedule()`, `_validate_formats()`, `_validate_host_active()`, `_validate_duplicate_pass()`, `_validate_visit_duration()`, `_sync_items_carried()`. See [validations.md](#).
- **on_submit()**: `generate_badge_number()` → `_generate_qr_code()` → `_send_approval_email()` (fail-soft) → `_notify_food_dept()` if meal flagged.
- **on_update()** when no longer Draft: `ensure_hospitality_request()` creates/updates the linked Hospitality Request.
- Workflow transitions write to **Visitor Event Log**.

Visitor Invitation

Purpose: Pre-visit invitation sent to a visitor with a tokenised portal link, so they can pre-fill their details before arrival.

Type: Document, naming VINV- .YYYY. - .#####

Key fields

Field	Type	Meaning
visitor_type	Select, required	Same options as Visitor Pass
visitor_email	Data, required	Recipient of the invitation email
visitor_mobile	Phone	Optional
host_employee	Link → Employee, required	Validated Active
visit_date	Date, required	
expected_checkin	Time, required	
expected_checkout	Time, required	
purpose_of_visit	Small Text	
meal_required / meal_type / conference_room	Hospitality fields the host can pre-set	
assigned_meal_slots	Small Text	Auto-derived from check-in/out window
invitation_status	Select	Draft / Sent / Opened / Saved / Submitted / Expired
invitation_token	Data, read-only	Secret URL token
portal_submission_url	Data, read-only	Full URL the visitor clicks
invitation_sent_on / link_opened_on / form_saved_on / form_submitted_on	Datetime	Telemetry
invitation_expires_on	Datetime	After this, the link 404s
visitor_pass	Link → Visitor Pass	Set once the visitor submits the form and a pass is created
reference_job_applicant	Link → Job Applicant	If invitation was triggered by HRMS Job Applicant offline interview

Side-effects

- `validate()` : visit date in future, email format, time range, host active.
- `send_invitation()` : generates token, builds portal URL, sends email.
- Portal submission via `portal.submit_pre_registration` creates / updates the linked Visitor Pass.

Security Log

Purpose: Records every gate event for a visitor — check-in, check-out, gate transfer, alert, badge collection. **Type:** Document, naming `SL - .YYYY. - .#####`

Key fields

Field	Type	Meaning
visitor_pass	Link → Visitor Pass, required	The pass being processed
event_type	Select, required	Check-In / Check-Out / Gate Transfer / Alert / Badge Collected
gate_name	Select, required	Sourced from VMS Settings gate_list
security_officer	Link → Employee	Filtered to users with Security role
verification_started_on	Datetime	When the officer began verifying
verification_duration	Duration	How long verification took
id_proof_match	Check, required for check-in	Confirms ID matches the person
pass_photo_match	Check, required for check-in	Confirms photo matches
photo_at_gate	Attach Image, required for check-in	Live gate photo
qr_code_scanned	Check	Whether the badge QR was scanned
temperature	Float	Body temperature if measured
symptoms_flag	Check	Any reported symptoms
symptoms_details	Small Text	What was reported
visited_area	Data	Required for Gate Transfer events
items_verification	Child Table → Security Item Verify	Per-item check at gate
exception_reason	Small Text	If verification was overridden
verification_notes	Small Text	Free-text comments

VIP-specific (auto-fetched from linked Visitor Pass for VIP visitors): | Field | Meaning | |---|---| | mdceo_notified | Was MD/CEO notified | | vip_meeting_room | The room | | vip_protocol_notes | Protocol info |

Side-effects

- validate() : enforces photo + identity match for check-in, area for gate transfer, pass-must-be-Approved.
- before_save() : writes a Visitor Event Log entry, calls sync_contact_trace() and sync_compliance_check() .
- Updates Visitor Pass.actual_checkin/checkout/status/current_location .

Hospitality Request

Purpose: Consolidates every hospitality arrangement (meal, cab, hotel, factory tour, buggy, greeting) into a single submittable request that goes through the Hospitality Manager for approval. **Type:** Document, naming pattern HOSP-{visitor_initials}-{ddmmyy} **Workflow:** [Hospitality Request Approval](#) (7 states)

Key fields

Linkage

Field	Type	Meaning
visitor_pass	Link → Visitor Pass, required	Whose hospitality this is for
visitor_name	Data, fetched	Pulled from VP
visitor_type	Data, fetched	Pulled from VP
visit_date	Date, fetched	Pulled from VP
visit_start_time	Datetime	First service slot
visit_end_time	Datetime	Last activity end
status	Select	Pending / In Progress / Completed / Cancelled (read-only mirror of workflow_state for reports)
workflow_state	Link → Workflow State	Real workflow state

Meal

Field	Type	Meaning
meal_required	Check	
meal_type	Select	Breakfast / Lunch / Dinner / All Day
assigned_meal_slots	Small Text	Auto-derived (e.g. "Lunch, Dinner")
hospitality_type	Select	Single Meal / Two Meals / Full Day
number_of_people	Int	Total head count
special_diet	Select	Vegetarian / Vegan / Jain / Halal / Non-vegetarian / Gluten Free
service_time	Datetime	When food is delivered
food_dept_staff_assigned	Link → Employee	Catering owner
food_status	Select	Pending / Ordered / Served / Completed / Cancelled
conference_room	Link → Conference Room	Where meal is served
seating_capacity	Int	Tested against room capacity

Cab

Field	Meaning
<code>cab_required</code>	Yes/no
<code>cab_type</code>	Pickup / Drop / Both
<code>pickup_location</code> / <code>pickup_datetime</code>	If type includes Pickup — both required
<code>drop_location</code> / <code>drop_datetime</code>	If type includes Drop — both required
<code>driver_name</code> / <code>cab_vendor</code> / <code>cab_booking_reference</code>	Logistics

Hotel

Field	Meaning
<code>hotel_required</code>	Yes/no
<code>hotel_name</code> / <code>hotel_room_type</code> / <code>booking_reference</code>	Booking details
<code>check_in</code> / <code>check_out</code>	Validated against the visit date window
<code>nights</code>	Auto-computed

Factory tour

Field	Meaning
<code>factory_tour_required</code>	Yes/no
<code>tour_date</code> / <code>tour_start_time</code> / <code>tour_end_time</code>	Window — must be inside visit window, end must be after start
<code>tour_guide</code>	Link → Employee
<code>tour_areas</code>	Child table → Factory Tour Area

Buggy / greeting

Field	Meaning
<code>buggy_required</code>	Yes/no — also auto-set when <code>factory_tour_required</code>
<code>buggy_pickup_point</code> / <code>buggy_datetime</code> / <code>buggy_driver</code> / <code>buggy_number</code>	Logistics — <code>buggy_number</code> conflicts checked
<code>greeting_required</code>	Yes/no — also auto-set for VIP
<code>greeting_type</code> / <code>greeting_delivery_time</code> / <code>greeting_assigned_to</code>	Logistics

Side-effects

- `validate()` : `_validate_visitor_pass_approved` gate (Δ can't move to Pending until VP is Approved), then `_compute_hotel_nights` , `_validate_cab_timing` , `_validate_tour_safety` ,

`_validate_buggy_conflict`, `_validate_seating_capacity`, `_validate_hotel_in_visit_window`, `_validate_activities_in_visit_window`.

- `populate_hospitality_request_from_pass()` — fetches meal/cab/hotel etc. from the linked Visitor Pass.
- Auto-created by `lifecycle.ensure_hospitality_request()` when a Visitor Pass is approved with `meal_required` / `cab_required` / `hotel_required` / `factory_tour_required` / `buggy_required` / `greeting_required` set.

Conference Room

Purpose: Master list of meeting rooms with capacity, location, and operating hours. **Type:** Document, naming `field:room_name` (the room name IS the primary key)

Key fields

Field	Type	Meaning
<code>room_name</code>	Data, required	E.g. "Sequoia Boardroom"
<code>room_type</code>	Select	Small Meeting Room / Medium Conference Room / Large Boardroom / Training Room / Video Conference Room
<code>is_active</code>	Check, default 1	Inactive rooms are hidden from booking dropdowns
<code>location</code>	Data, required	Building / branch (e.g. "Block A — Executive Wing")
<code>floor</code>	Data	Floor reference (e.g. "3rd Floor")
<code>capacity</code>	Int, required	Max seating
<code>available_from</code> / <code>available_to</code>	Time	Operating hours, default 08:00 - 20:00
<code>min_booking_minutes</code>	Int	Minimum booking duration
<code>max_booking_hours</code>	Int	Maximum booking duration
<code>amenities</code>	Small Text	Free text — e.g. "75-inch 4K display, VC bar, whiteboard"
<code>description</code>	Small Text	Use-case notes

Conference Room Booking

Purpose: A time-slot reservation against a Conference Room. **Type:** Document, submittable, naming series `CRB-.YYYY.-.#####` **Workflow:** [Conference Room Booking Approval](#) (5 states)

Key fields

Field	Type	Meaning
meeting_title	Data, required	What the meeting is called
conference_room	Link → Conference Room, required	Which room
meeting_type	Select, required	Internal / External / Hybrid (External/Hybrid auto-set service flags)
booking_date	Date, required	Cannot be in the past
start_time / end_time	Time, required	start < end; both required
duration_hours	Float, computed	Auto-calculated
booked_by	Link → Employee, required	Who reserved it
expected_attendees	Int, required	≤ room capacity
visitor_pass	Link → Visitor Pass	Optional — if booking is for a visitor meeting (gate fires if linked VP isn't Approved)
room_cleaning_required / water_required / coffee_tea_required	Check	Auto-set for External/Hybrid meetings
status	Select	Mirror of workflow state
workflow_state	Link → Workflow State	

Side-effects

- `validate()` : schedule sanity, capacity check, time-overlap detection (no double-booking the same room/time), operating-hours check, visitor-pass-approved gate.
- `on_submit()` : status → Approved.
- `on_cancel()` : status → Cancelled.
- Whitelisted helpers `get_available_rooms` , `get_room_schedule` , `get_booking_events` power the calendar/booking UI.

Visitor Blacklist

Purpose: Prevent specific visitors from being granted a pass — auto-checked against new pass creations. **Type:** Document

Key fields

Field	Meaning
visitor_name	Display name
id_proof_type / id_proof_number	Either ID OR (name + type) is required for identification
reason	Mandatory free-text — audit/compliance requirement
blocked_by	Link → User, defaults to current session user
blocked_on	Date, defaults to today
is_active	Check — duplicate active block on same ID is rejected

Side-effects

- `validate()` : enforces reason mandatory, identification mandatory, no duplicate active block.
- A new Visitor Pass with a matching `id_proof_number` of an active blacklist entry triggers a security alert email and is blocked.

Visitor Event Log

Purpose: Audit trail — one row per important transition on a Visitor Pass. **Type:** Document, naming hash

Key fields

Field	Meaning
visitor_pass	Link → Visitor Pass
event_type	Submitted / Approved / Rejected / Checked-In / Checked-Out / Gate Transfer / Item Verified / etc.
event_status	Pending / Recorded / Failed
source_doctype / source_name	What doc raised the event (Security Log, Hospitality Request, etc.)
details	JSON dump of the event context
event_time	Auto-set
user	Who triggered it

Read-only for everyone except System Manager. Used by Compliance Check and reports.

Compliance Check

Purpose: Auto-built per-visitor scorecard summarising whether all gate-side requirements were met. **Type:** Document

Key fields

Field	Meaning
visitor_pass	The pass being audited
compliance_status	Compliant / Needs Review / No Show
score	0-100 (subtracts 15 per missing requirement)
id_verified / pass_photo_verified / gate_photo_captured / items_verified / hospitality_closed / no_show	Boolean checks
verification_duration	How long check-in took
current_location	Last known location
missing_requirements	Newline-joined list of gaps
exception_reason	If verification was overridden
last_security_log	Link to the security log used to compute
last_checked_on	Timestamp

Auto-created/updated by `sync_compliance_check()` after every Security Log save.

Contact Trace Record

Purpose: Tracks where each visitor moved during their visit so contact-tracing reports can be generated for health incidents. **Type:** Document

Key fields

Field	Meaning
visitor_pass	Link → Visitor Pass
security_log	Link → Security Log that triggered this record
visited_area	The area at this point in time
time_in / time_out	Datetimes
status	Active / Closed
exposure_risk	Low / High (auto-derived from temperature $\geq 37.5^{\circ}\text{C}$ or symptoms_flag)
notes	Free text

Created/updated by `sync_contact_trace()`.

Emergency Event

Purpose: An evacuation drill, fire alarm, gas leak, or any incident requiring all visitors on premises to be accounted for.

Type: Document

Key fields

Field	Meaning
event_type	Fire / Gas Leak / Drill / Evacuation / Other
event_datetime	When it started
assembly_point	Where everyone goes
status	Active / Closed
notes	Incident details
closed_on	When the all-clear was given

Side-effects

- When `status = Active`, `lifecycle.generate_emergency_muster_records()` builds an Evacuation Muster row for **every Checked-In visitor**, prepopulating the assembly point.

Evacuation Muster

Purpose: One row per visitor-still-on-premises during an Emergency Event, tracking whether they've reached the assembly point. **Type:** Document

Key fields

Field	Meaning
emergency_event	Link → Emergency Event
visitor_pass	Link → Visitor Pass
employee_host	Link → Employee (the host)
assembly_point	Copied from event
last_known_location	From contact trace
accounted_status	Pending / Accounted / Missing / Excused
accounted_time	When marked Accounted
notes	Incident notes

Used by the "Missing in Muster" number card on the Security Dashboard.

VMS Settings

Purpose: App-wide configuration. Single doctype. **Type:** Single

Key fields

Field	Meaning
admin_email	Operations admin contact
food_dept_email	Catering team email
default_checkout_time	Time used to populate expected_checkout if not specified
max_visit_duration_hrs	Cap for visit duration; passes longer are rejected unless invitation-backed
require_visitor_photo	If 1, visitor_photo cannot be empty on save
qr_scan_required_at_gate	If 1, Security Log requires qr_code_scanned=1 for check-in
require_item_declaration	If 1, at least one Visitor Item row required before workflow Submit
block_check_in_without_verification	If 1, refuses Check-In without ID + photo match
blacklist_action	Block Entry / Alert Only — what to do on blacklist match
gate_list	Newline-separated list of gate names — feeds the Security Log gate dropdown
enable_badge	If 1, badge numbers auto-generate on approval
badge_required_for	Newline list of visitor types that get a badge (default: all 5)
badge_prefix_<type>	3-char prefix per visitor type (e.g. CUS, CON, CAN, SUP, VIP)
badge_colour_<type>	Display colour per type (Green / Orange / Purple / Teal / Gold)

Visitor Item

Purpose: One row per item a visitor declared they're carrying. **Type:** Child table on Visitor Pass. visitor_items

Key fields

Field	Meaning
item_name	E.g. "Dell laptop"
item_category	Electronics / Tool / Document / Sample / Gift / Perishable / Weapon / Other
quantity	Default 1
serial_number	Optional
description	Free text
status	Pending / Verified / Exception
verified_by_security	Check, set by Security Log → Security Item Verify
verification_remarks	Free text

Security Item Verify

Purpose: Per-item verification record produced when Security checks the visitor's items at the gate. **Type:** Child table on Security Log. `items_verification`

Key fields

Field	Meaning
<code>item_name</code>	Item being verified
<code>verified</code>	Check
<code>notes</code>	Free text

When a check-in's `items_verification` rows are saved, the corresponding Visitor Item rows on the parent pass are updated.

Factory Tour Area

Purpose: Areas the visitor will pass through during a factory tour — used for safety briefing checklist. **Type:** Child table on Hospitality Request. `tour_areas`

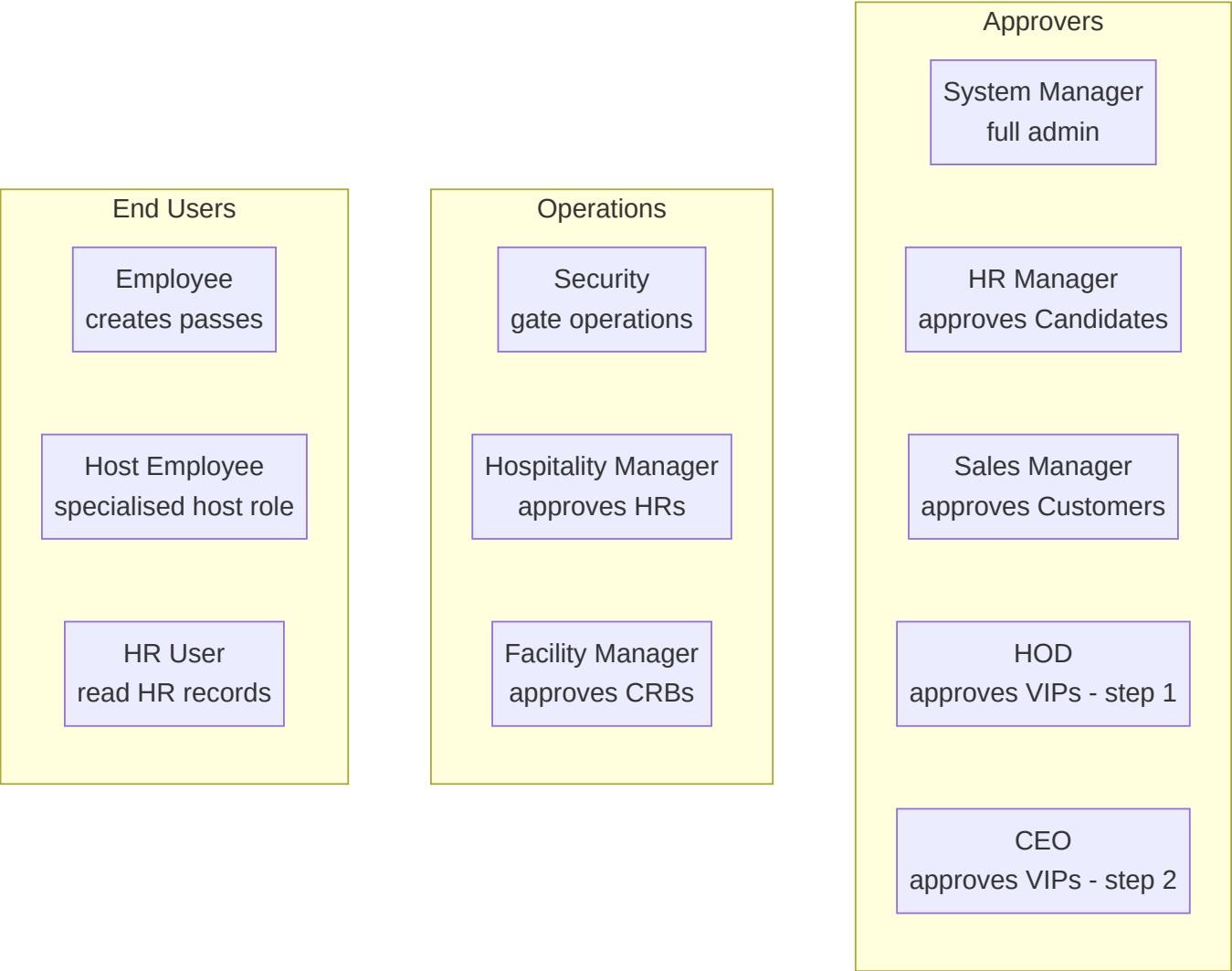
Key fields

Field	Meaning
<code>area_name</code>	E.g. "Assembly Line 2", "QC Lab"
<code>safety_briefing_required</code>	Check
<code>ppe_required</code>	Free text — what protective gear is needed
<code>notes</code>	Free text

Roles & Permissions

The app uses **11 distinct roles** to gate every action. Each role is a single responsibility — combine them on a user as needed (e.g. an Operations head might hold both `System Manager` and `Hospitality Manager`).

The 11 roles



Role	Responsibility	Lane
System Manager	Full admin; approves Contractor + Supplier passes	Visitor Pass: Contractor, Supplier
HR Manager	Approves Candidate visitor passes	Visitor Pass: Candidate
Sales Manager	Approves Customer visitor passes	Visitor Pass: Customer
HOD	Approves VIP visitor passes — first step	Visitor Pass: VIP step 1
CEO	Approves VIP visitor passes — second step (after <code>mdceo_notified</code>)	Visitor Pass: VIP step 2
Security	Files Security Log entries (check-in, check-out, gate transfer); read-only on Visitor Pass	Security Log + read-only Visitor Pass
Hospitality Manager	Approves Hospitality Requests; manages catering / cab / hotel staff	Hospitality Request workflow
Facility Manager	Approves Conference Room Bookings	Conference Room Booking workflow
Host Employee	Same as Employee but optimised UX for hosts who frequently invite visitors	Same as Employee
Employee	Creates Visitor Passes / Visitor Invitations / Hospitality Requests / Conference Room Bookings as a host	Submitter side of all workflows
HR User	Standard ERPNext role — required for Job Applicant integration; can read HR records	Read-only HR side

Per-doctype role-level permission matrix

The matrix below is the **role-level grant** (`frappe.permissions.has_permission(doctype, ptype, user)` without a doc filter). Per-document filters (e.g. "Sales Manager only sees Customer passes") are layered on top via `permissions.py` Python hooks.

Doctype	Sys Mgr	HR Mgr	Sales Mgr	HOD	CEO	Security	Hosp Mgr	Fac Mgr	Host Emp	Employee	HR User
Visitor Pass	RWCS	RWCS	RWCS	RWS	RWS	R	–	–	RWCS	RWCS	–
Visitor Invitation	RWCS	RWCS	RWCS	–	–	–	RWC	RWC	RWC	RWC	–
Security Log	RWCS	–	–	–	–	RWC	–	–	–	–	–
Hospitality Request	RWCS	–	–	–	–	–	RWCS	–	RWCS	RWC	–
Conference Room Booking	RWCS	–	–	–	–	–	–	RWCS	RWC	RWC	–
Conference Room	RWCS	–	–	–	–	–	R	RWC	R	R	–
Visitor Blacklist	RWCS	–	–	–	–	–	–	–	–	–	–
Visitor Event Log	RWCS	–	–	–	–	–	–	–	–	–	–
Compliance Check	RWCS	–	–	–	–	–	–	–	–	–	–
Contact Trace Record	RWCS	–	–	–	–	–	–	–	–	–	–
Emergency Event	RWCS	–	–	–	–	–	–	–	–	–	–
Evacuation Muster	RWCS	–	–	–	–	–	–	–	–	–	–
VMS Settings	RW	–	–	–	–	–	–	–	–	–	–
Page (Frappe core)	R	–	–	–	–	–	–	–	–	R	–

Legend: R ead, W rite, C reate, S ubmit, – no access.

Two-layer permission system

1. **Role-level** (the table above) — decided by DocPerm rows in each doctype's JSON. Frappe checks first.
2. **Per-document Python hook** for Visitor Pass — implemented in [permissions.py](#) . Filters by:
 - **Owner**: the user always sees passes they created.
 - **Host**: the user sees passes where they are `person_to_visit` .
 - **Lane**: each approver only sees the visitor types they own.
 - System Manager → Contractor + Supplier

- HR Manager → Candidate
- Sales Manager → Customer
- HOD / CEO → VIP
- Security → only `status` in (Approved, Items Verified, Checked-In, Checked-Out) — and only **read** (write/submit/cancel are blocked even if the role has them)
- **Cross-cutting read** for Hospitality Manager / Facility Manager: not in the current code (they see only passes they own/host on the list); a previous patch added blanket-read but it was reverted.

Workflow-level permission

Each workflow restricts who can run each transition (`allowed` field per transition). The role-level perm table determines whether a user can *open* a doc; the workflow determines whether they can *advance* it. See [workflows.md](#).

What each user sees in their list views (live data on the demo site)

User / Role	Visitor Pass	Visitor Invitation	Security Log	Hospitality Req	Conf. Booking	Conf. Room	Blacklist
Administrator (Sys Mgr)	129	20	31	81	34	7	5
HOD	25 (VIP only)	20	⊘	81	34	7	⊘
CEO	25 (VIP only)	20	⊘	81	34	7	⊘
Sales Manager	25 (Customer only)	20	⊘	81	34	7	⊘
HR Manager	25 (Candidate only)	20	⊘	81	34	7	⊘
Security	56 (approved+ only)	20	31	81	34	7	⊘
Hospitality Manager	only owned/hosted	20	⊘	81	34	7	⊘
Facility Manager	only owned/hosted	20	⊘	81	34	7	⊘
Pure Employee	only owned/hosted	20	⊘	81	34	7	⊘

⊘ = HTTP 403 / no role-level read.

Adding a user

1. Go to **Users** at `/app/user/new` . Enter email, name, set `User Type` = `System User` .
2. Add the desired role(s) to the user's **Roles** child table.
3. Save the user.
4. **Provision an Employee record** at `/app/employee/new` linked to this user (`user_id` field). Without this, Frappe's `Employee` role gets stripped on the next User save, and the user can't act as a host.

5. The `ensure_visitor_management_roles.py` patch handles this auto-provisioning for Security, Hospitality Manager, Facility Manager users on every `bench migrate`.
6. For end-to-end usability, the host Employee should also have a **Department** so the Visitor Pass can populate `host_department` automatically.

Common tasks per role

Role	Typical day-to-day actions
Employee / Host	Open <code>/app/visitor-pass/new</code> , fill in visitor details, save → click Submit for Approval . Also: book conference rooms via <code>/app/conference-room-booking/new</code> .
Sales Manager	Filter Visitor Pass list by <code>workflow_state = Pending Sales Manager</code> → review → click Approve or Reject .
HR Manager	Same as Sales Manager but lane = <code>Pending HR Manager</code> .
System Manager	Same for Contractor + Supplier; also manages VMS Settings, Conference Rooms, Visitor Blacklist.
HOD	VIP passes filtered by <code>Pending HOD</code> ; on Approve they move to <code>Pending CEO</code> .
CEO	VIP passes filtered by <code>Pending CEO</code> ; tick <code>mdceo_notified</code> then Approve.
Security	Visitor Pass list shows only Approved+; at gate, fills out a Security Log per visitor (Check-In, then Check-Out later).
Hospitality Manager	Hospitality Request list filtered by <code>Pending Manager Approval</code> ; approves, then marks <code>Start Work</code> when prep begins, <code>Complete</code> when delivered. Receives daily 7 AM digest email.
Facility Manager	Conference Room Booking list filtered by <code>Pending Approval</code> ; reviews overlaps, approves. Also creates new Conference Rooms.

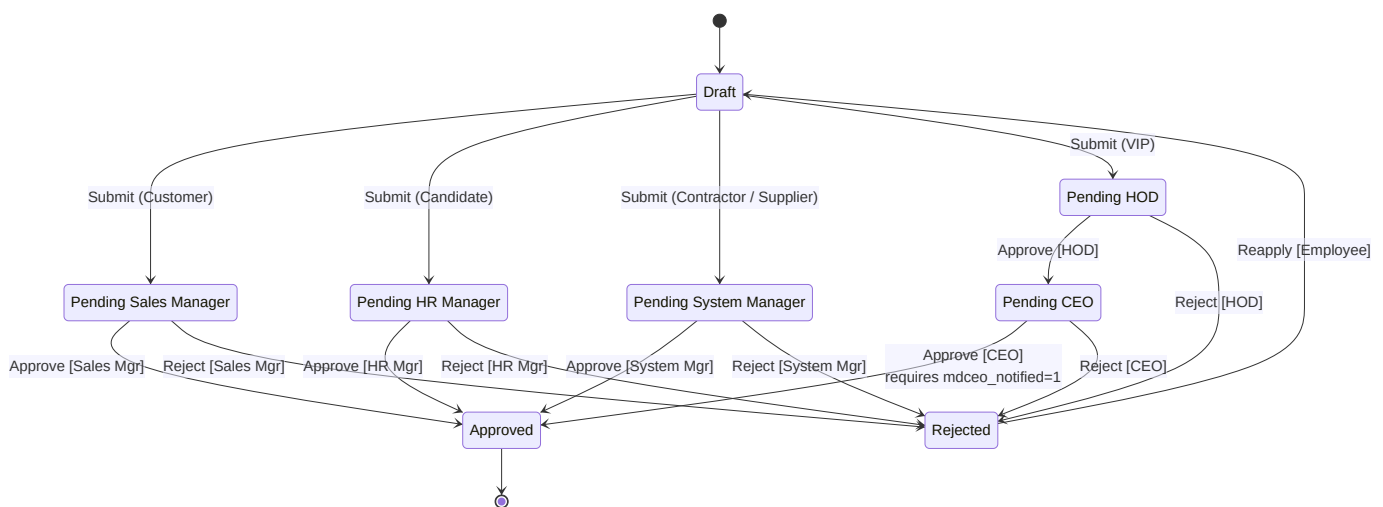
Workflows

Three workflows govern approvals in the app. Each is defined in `fixtures/workflow.json` and re-installed on every `bench migrate`.

Visitor Pass Approval

The headline workflow — controls every visitor's path from request to gate.

State diagram



State table

State	doc_status	Edit role	Means
Draft	0	Employee	Pass is being filled — visitor not yet submitted for approval
Pending Sales Manager	0	Sales Manager	Customer pass awaiting Sales Manager review
Pending HR Manager	0	HR Manager	Candidate pass awaiting HR Manager review
Pending System Manager	0	System Manager	Contractor / Supplier pass awaiting System Manager review
Pending HOD	0	HOD	VIP pass — first approval needed from HOD
Pending CEO	0	CEO	VIP pass approved by HOD, awaiting CEO sign-off
Approved	1	System Manager	Pass is live; QR + badge generated; visitor can present at gate
Rejected	0	Employee	Pass turned down; can be edited and re-submitted

Routing rule (auto)

The `_align_workflow_lane_with_visitor_type()` validator on Visitor Pass routes the `Submit` action to the correct lane based on `visitor_type`:

```
PENDING_LANES_BY_VISITOR_TYPE = {
  "Contractor": ("Pending System Manager",),
  "Supplier":   ("Pending System Manager",),
  "Customer":   ("Pending Sales Manager",),
  "Candidate":  ("Pending HR Manager",),
  "VIP":        ("Pending HOD", "Pending CEO"), # two-step
}
```

A user holding the wrong lane's role for a given pass cannot approve it — they're filtered out by both `permission_query_conditions` and the per-doc `has_visitor_pass_permission` hook.

VIP CEO gate

Before CEO can run **Approve** on a Pending CEO VIP pass, the field `mdceo_notified` must be `1`. Otherwise `before_submit` raises:

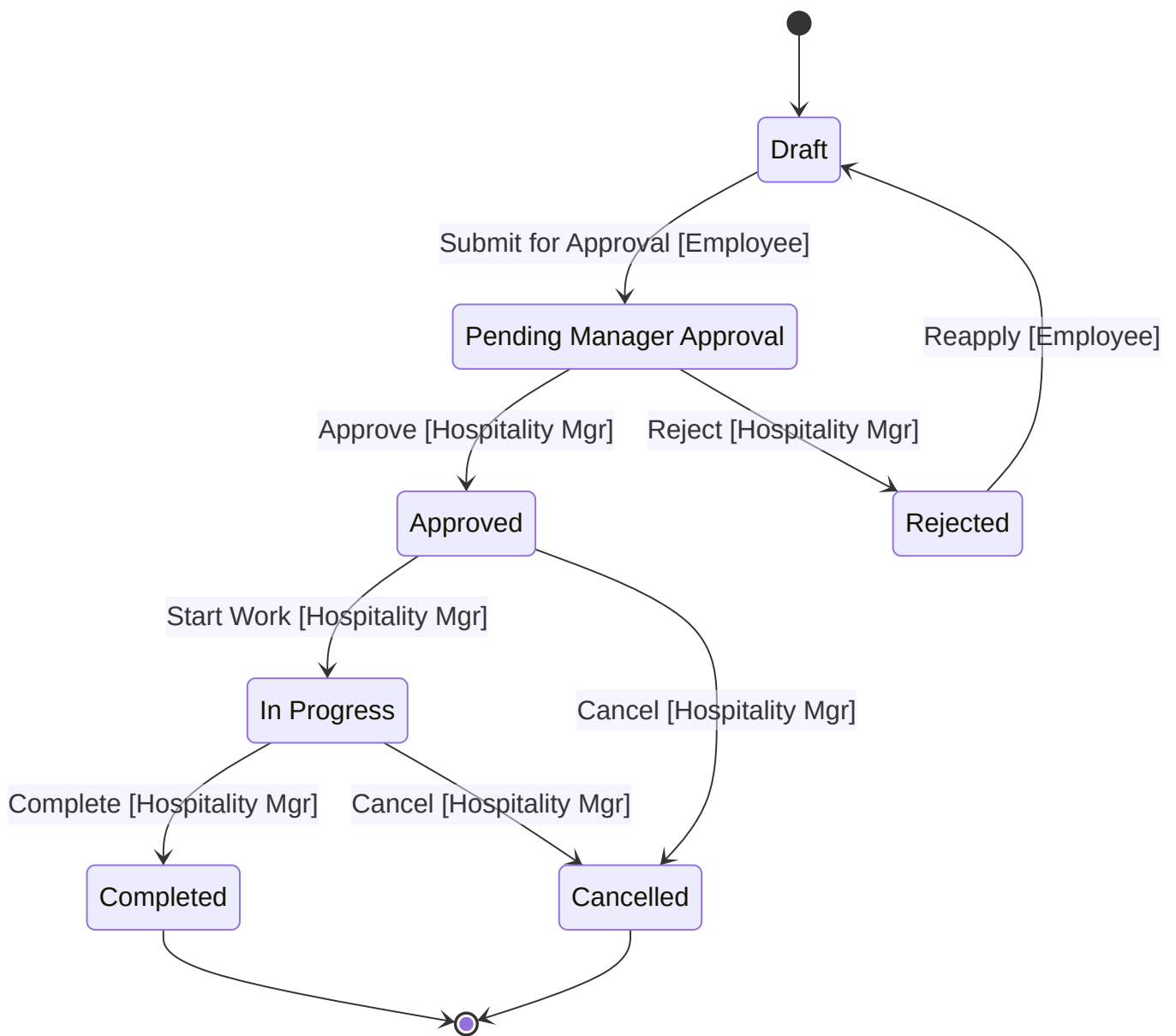
MD/CEO must be notified before submitting a VIP Visitor Pass. Please check the MD/CEO Notified field in the VIP Details section.

This is a real-world rule — the company's MD/CEO is told a high-profile guest is arriving before the workflow lets them be formally approved.

Hospitality Request Approval

Controls the lifecycle of every hospitality bundle (meal + cab + hotel + tour + buggy + greeting).

State diagram



State table

State	doc_status	Edit role	Means
Draft	0	Employee	Request being prepared (often auto-created by Visitor Pass approval)
Pending Manager Approval	0	Hospitality Manager	Request submitted, awaiting Hospitality Manager approval
Approved	0	Hospitality Manager	Approved; arrangements can begin
In Progress	0	Hospitality Manager	Work has started (catering ordered, cab assigned, etc.)
Completed	0	Hospitality Manager	All hospitality delivered; closes the loop
Rejected	0	Employee	Manager refused — needs rework, re-applies to Draft
Cancelled	0	Hospitality Manager	Cancelled (visitor cancelled, not needed)

⚠ Visitor-Pass-Approved gate

You **cannot** move a Hospitality Request out of Draft (i.e. submit for approval) unless the linked Visitor Pass's `status` is one of `Approved` / `Items Verified` / `Checked-In` / `Checked-Out`.

Real-world reason: hospitality preparation (booking a cab, ordering catering, blocking a hotel room) costs money. We don't spend it until the visitor is confirmed.

Implemented in `HospitalityRequest._validate_visitor_pass_approved()`. Error message:

Cannot send Hospitality Request HOSP-XYZ-070526 for approval — the linked Visitor Pass VP-2026-NNNNN is still Draft. Hospitality preparation can only begin once the visitor is confirmed (Visitor Pass must be Approved).

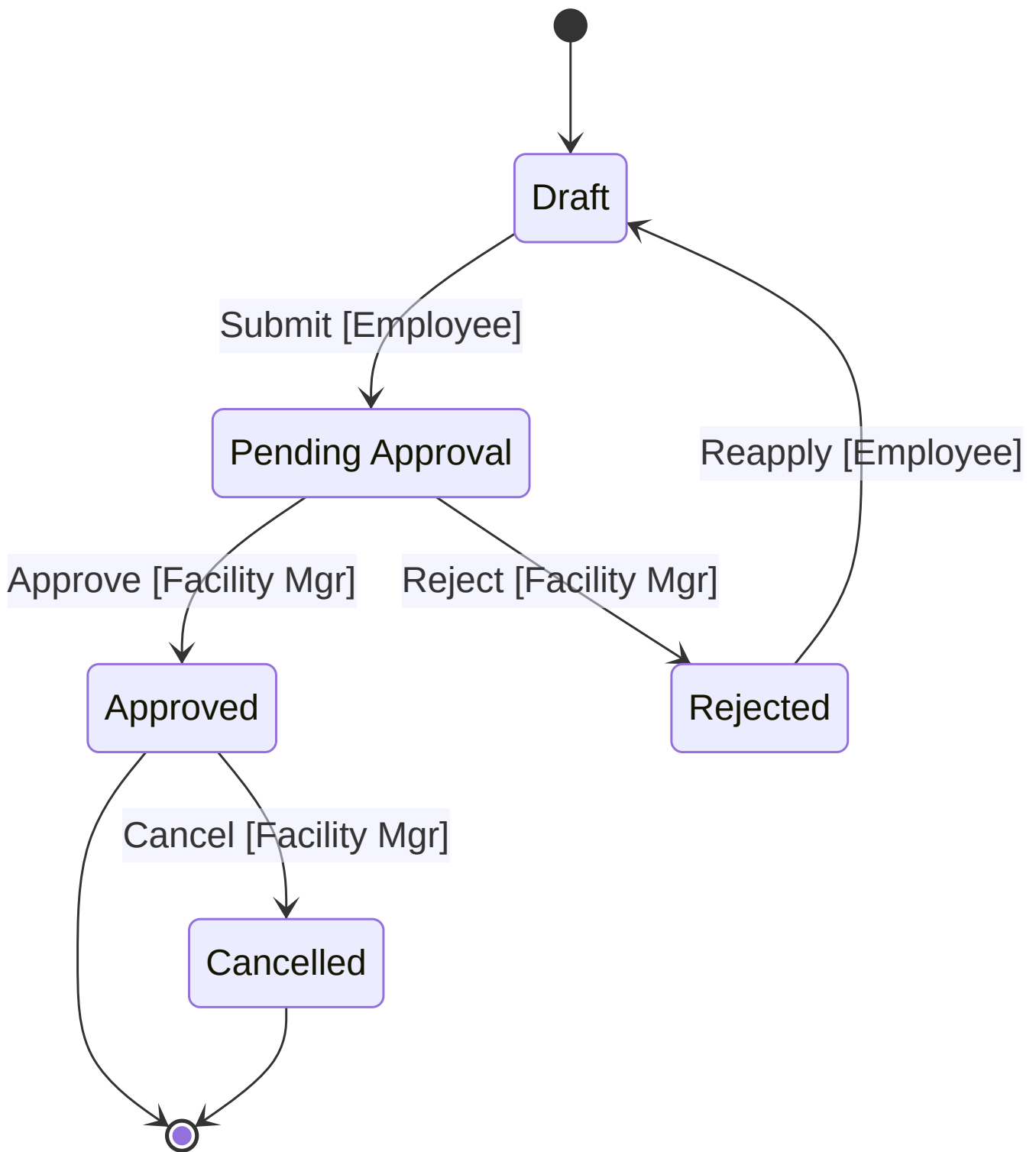
Auto-creation

When a Visitor Pass with `meal_required=1` (or any other hospitality flag set) is approved, `lifecycle.ensure_hospitality_request()` automatically creates a Hospitality Request linked to the pass with the right fields pre-populated. The Hospitality Manager sees it pop up in their queue without any human routing.

Conference Room Booking Approval

Controls the lifecycle of every meeting room reservation.

State diagram



State table

State	doc_status	Edit role	Means
Draft	0	Employee	Booking being created
Pending Approval	0	Facility Manager	Booking awaiting Facility Manager review
Approved	1	Facility Manager	Booking confirmed; room is reserved
Rejected	0	Employee	Refused — can be edited and re-applied
Cancelled	2	Facility Manager	Approved booking later cancelled

⚠ Visitor-Pass-Approved gate

Same rule as Hospitality Request — a Conference Room Booking with a `visitor_pass` link cannot move to Pending Approval until that pass is Approved.

Internal-only meetings (no `visitor_pass` link) are unaffected — they go through the workflow normally.

Validations on every save

The booking enforces: - `validate_schedule()` — past dates, time order, both times required - `validate_capacity()` — `expected_attendees` ≤ room capacity - `validate_overlap()` — no double-booking of the same room/time slot - `validate_operating_hours()` — must fall within the room's `available_from` / `available_to` - `auto_set_service_flags()` — for External / Hybrid meetings: set `room_cleaning_required`, `water_required`, `coffee_tea_required`

Modifying a workflow

To change states / transitions / approver roles:

1. Edit the relevant patch: - **Visitor Pass** → `patches/post_model_sync/sync_visitor_pass_workflow.py` - **Hospitality Request** → `patches/post_model_sync/sync_hospitality_request_workflow.py` - Conference Room Booking → loaded from `fixtures/workflow.json` directly
2. Run the patch: `bench --site <site> execute visitormanagement.patches.post_model_sync.<name>.execute`
3. Or just `bench --site <site> migrate` and the patch runs as part of the cycle.

The patches are **idempotent** — they upsert the workflow rather than recreating it, so existing in-flight docs aren't disturbed.

⚠ **Keep the role mapping in sync:** if you add a new pending state for a new visitor type, also update `PENDING_LANES_BY_VISITOR_TYPE` in `visitor_pass.py` so the routing validator doesn't reject the Submit action.

Business Flows

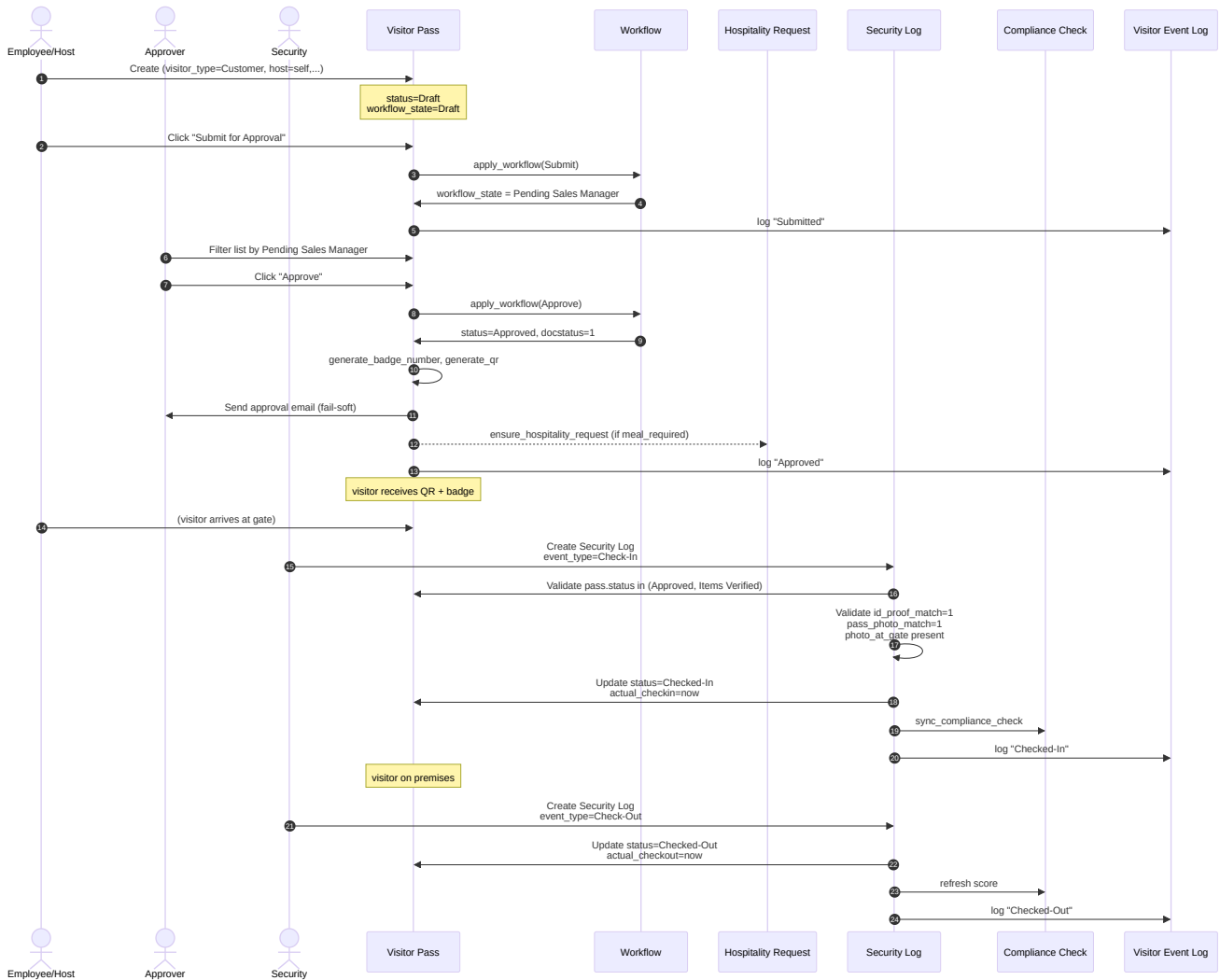
End-to-end sequences for every major scenario in the app, with the actors, doctypes, and side-effects shown explicitly.

Index

1. [Visitor Pass Lifecycle \(the headline flow\)](#)
 2. [Hospitality Request Flow](#)
 3. [Conference Room Booking Flow](#)
 4. [Gate Operations / Security Log](#)
 5. [Pre-Registration via Portal](#)
 6. [Job Applicant → Visitor Invitation \(HRMS integration\)](#)
 7. [Emergency Evacuation](#)
 8. [Visitor Blacklist Match](#)
 9. [Daily Hospitality Digest](#)
-

1. Visitor Pass Lifecycle

The headline flow. From the moment a host wants to invite a visitor, all the way through approval, gate check-in, and post-visit compliance.

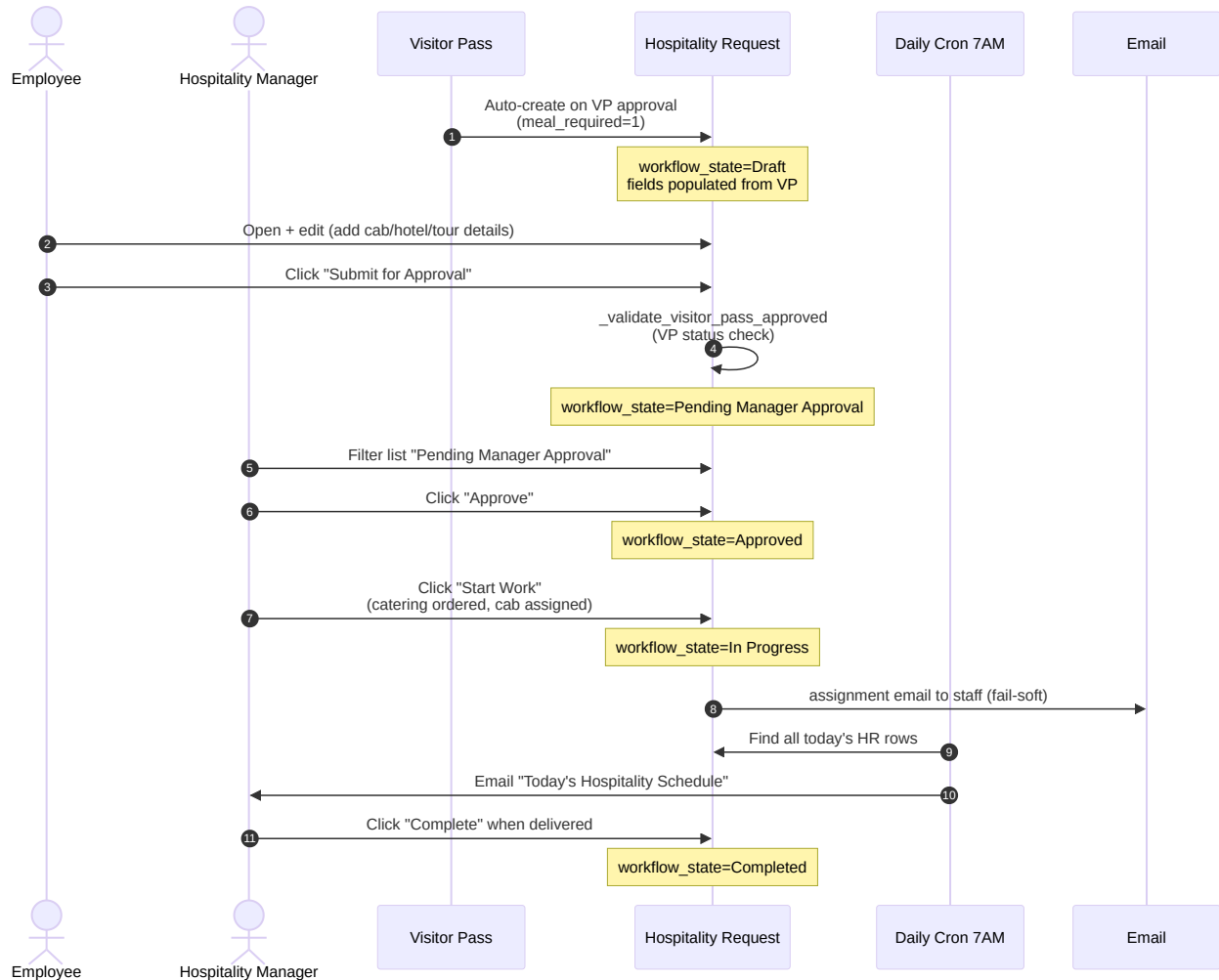


Step-by-step

1. **Host creates** the Visitor Pass with the visitor's identity, photo, ID proof, and visit details.
2. **Host submits** for approval; the workflow routes to the right lane based on `visitor_type`.
3. **Approver** opens the filtered list (Pending Sales Manager , Pending HR Manager , etc.) and approves.
4. On approval: badge auto-generates (CUS-2026-XXXXX), QR code embedded in approval email.
5. If `meal_required` (or any other hospitality flag) is set, a **Hospitality Request** is auto-created in Draft state.
6. **Visitor arrives at gate**. Security verifies identity (ID + photo match) and captures gate photo.
7. Security files a **Check-In Security Log** → pass status flips to **Checked-In**.
8. When visitor leaves: another Security Log with **Check-Out** → pass status flips to **Checked-Out**.
9. After every Security Log save, `sync_compliance_check` updates the Compliance Check scorecard.

2. Hospitality Request Flow

Auto-created when a Visitor Pass is approved with hospitality flags set; Hospitality Manager approves and tracks delivery.



The visitor-pass-approved gate [△](#)

If the linked Visitor Pass is still `Draft` , the **Submit for Approval** action raises:

Cannot send Hospitality Request HOSP-XYZ-NNNNNN for approval — the linked Visitor Pass VP-2026-NNNNN is still Draft. Hospitality preparation can only begin once the visitor is confirmed (Visitor Pass must be Approved).

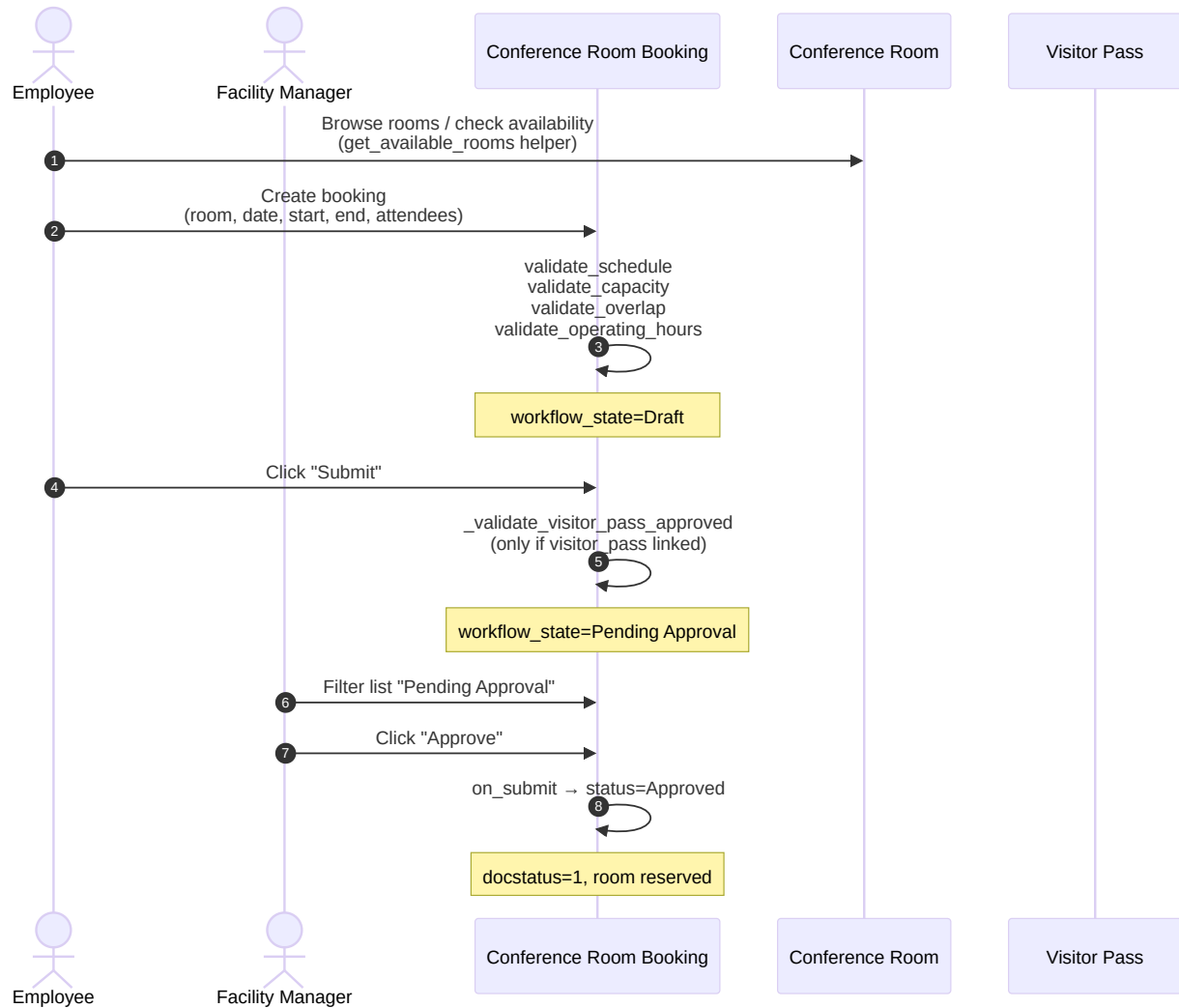
This is the real-world rule: don't pay for catering for a visitor who hasn't been approved.

Daily 7 AM digest

`tasks.send_daily_hospitality_digest()` runs every day at 07:00 (cron `0 7 * * *`). It builds an HTML summary of every Hospitality Request scheduled for today and emails: - Hospitality Manager - Hospitality User - Transport Coordinator - Front Office Executive - Factory Tour Coordinator - Greeting Staff

Sections in the email: **Cabs**, **Hotel check-ins**, **Factory tours**, **Buggy requests**, **Greeting assignments** — each with status icons (Pending / In Progress / Completed) and links back to the Hospitality Request.

3. Conference Room Booking Flow



Validation guards

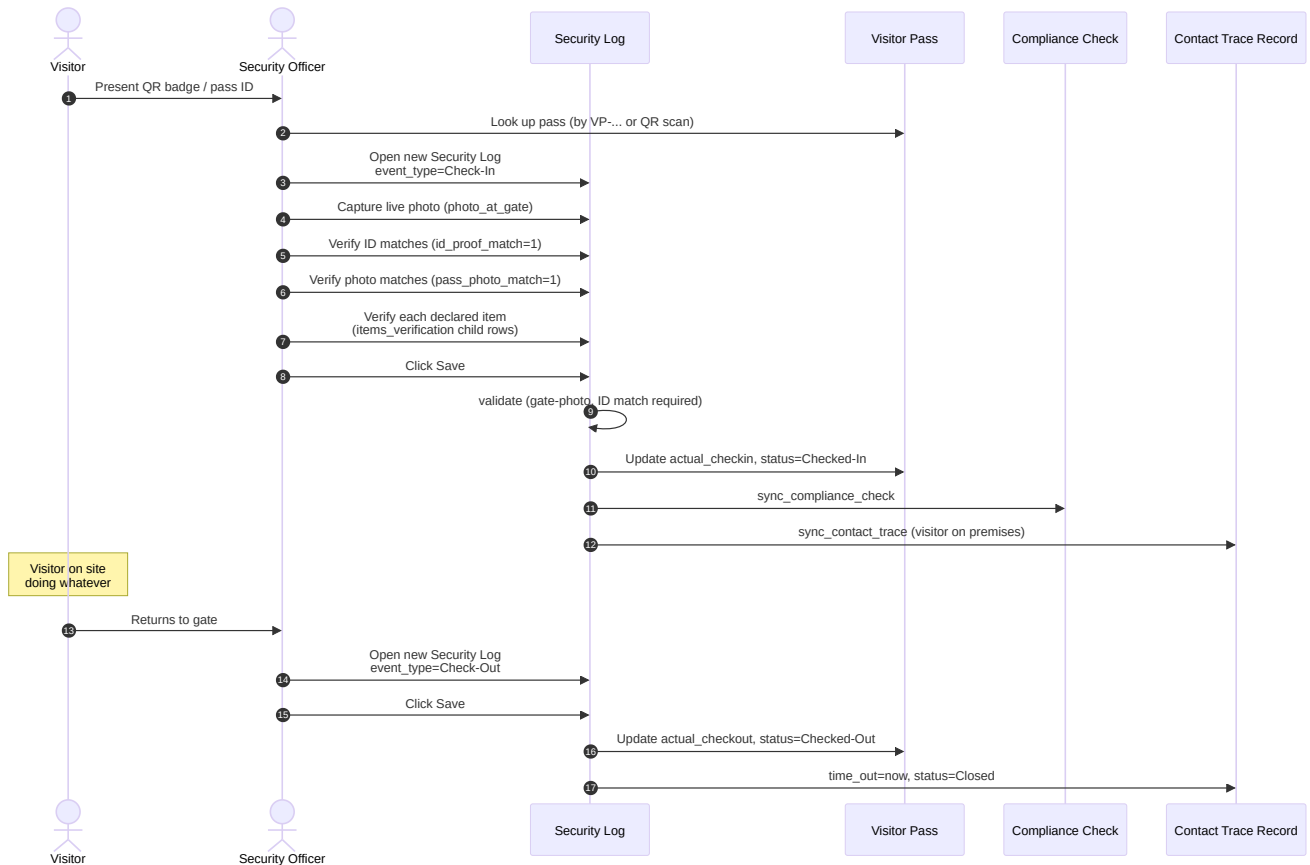
- **Past date** → rejected
- **Start ≥ End** → rejected
- **Attendees > room capacity** → rejected
- **Time overlap** with another booking → rejected
- **Outside room operating hours** → rejected
- **VP-not-Approved gate** (only if `visitor_pass` linked) → rejected on Submit

Internal vs. external meetings

- `meeting_type = External` or `Hybrid` → auto-sets `room_cleaning_required`, `water_required`, `coffee_tea_required`.
- `meeting_type = Internal` → no auto-flags; lighter approval path.

4. Gate Operations / Security Log

Security desk's day-to-day flow.

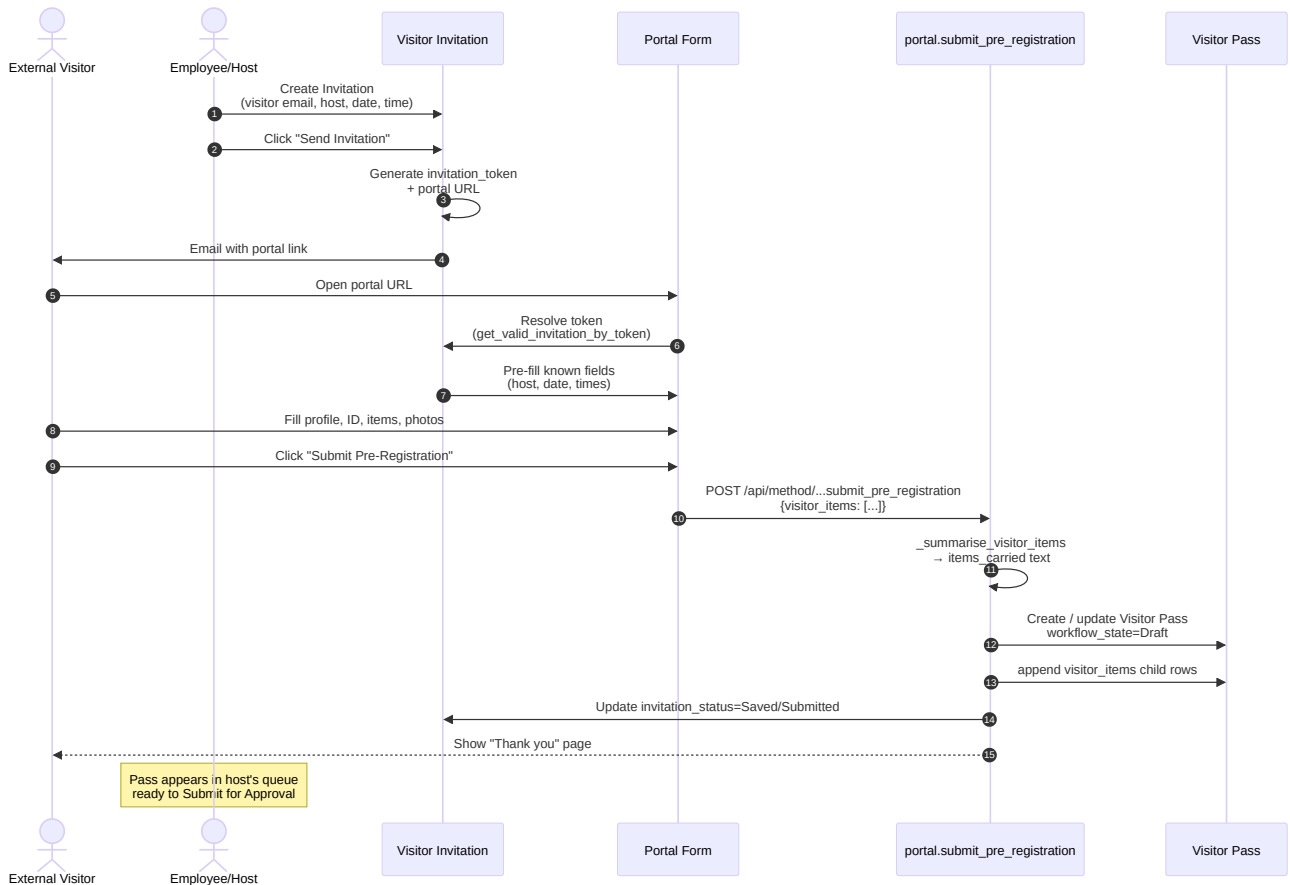


Validations on every Security Log save

- Linked Visitor Pass must be Approved or beyond
- Cannot Check-In an already-Checked-In visitor
- Cannot Check-Out a visitor that hasn't Checked-In
- Check-In : requires photo_at_gate , id_proof_match=1 , pass_photo_match=1
- Gate Transfer : requires visited_area

5. Pre-Registration via Portal

Visitor fills the public web form (with a tokenised invitation link) and the system creates a Visitor Pass behind the scenes.



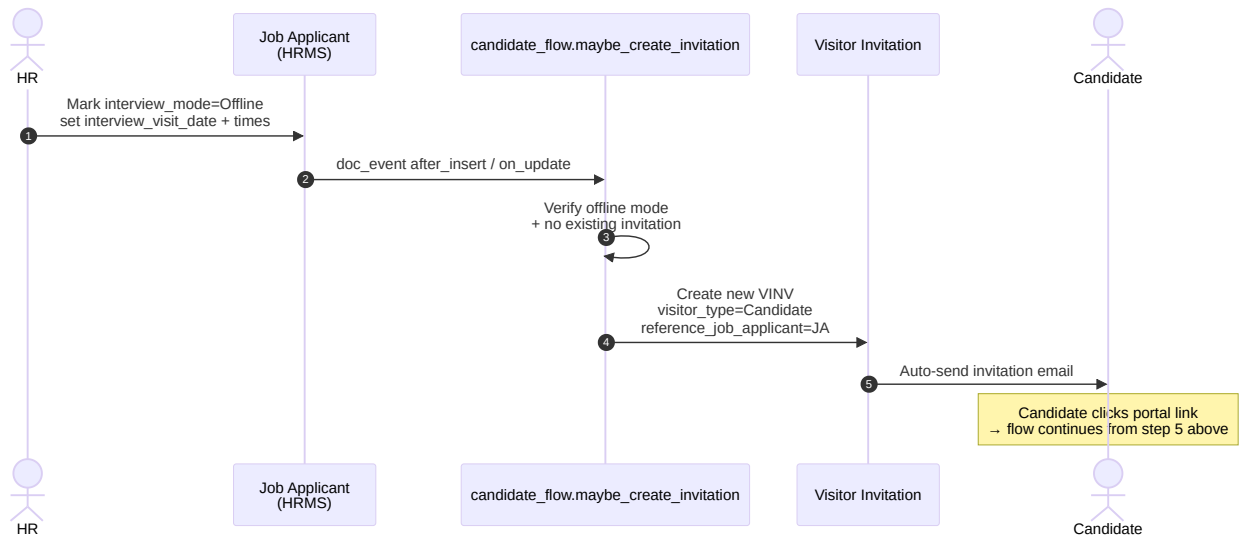
Items capture (single source of truth)

The portal uses ONE structured "Visitor Items" section (JS-injected). The visitor lists each item carrying once; the controller: - Creates one `Visitor Item` child row per entry on the resulting Visitor Pass. - Auto-derives `items_carried` (free-text summary like *"Dell XPS Laptop, Sandisk USB Drive (x2), Toolkit"*) for the printable badge.

The `_sync_items_carried` controller hook preserves the multi-row data — it doesn't collapse the rich rows down to one mirror row when the summary already matches what the portal sent.

6. Job Applicant → Visitor Invitation

Automatic flow: when an HRMS Job Applicant is marked for an offline interview, a Visitor Invitation is auto-created and sent.



Triggered by `hooks.py` :

```

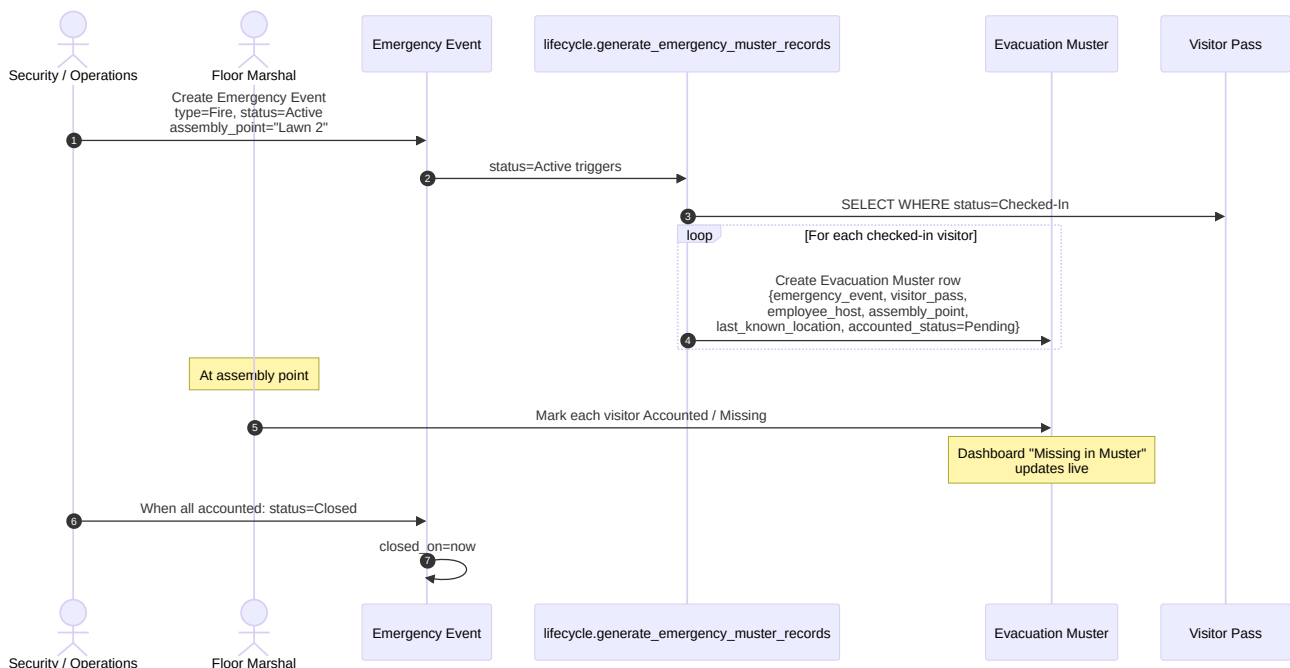
doc_events = {
    "Job Applicant": {
        "after_insert": "...candidate_flow.maybe_create_invitation",
        "on_update": "...candidate_flow.maybe_create_invitation",
    }
}

```

Failure to send the invitation email is logged but doesn't block the Job Applicant save.

7. Emergency Evacuation

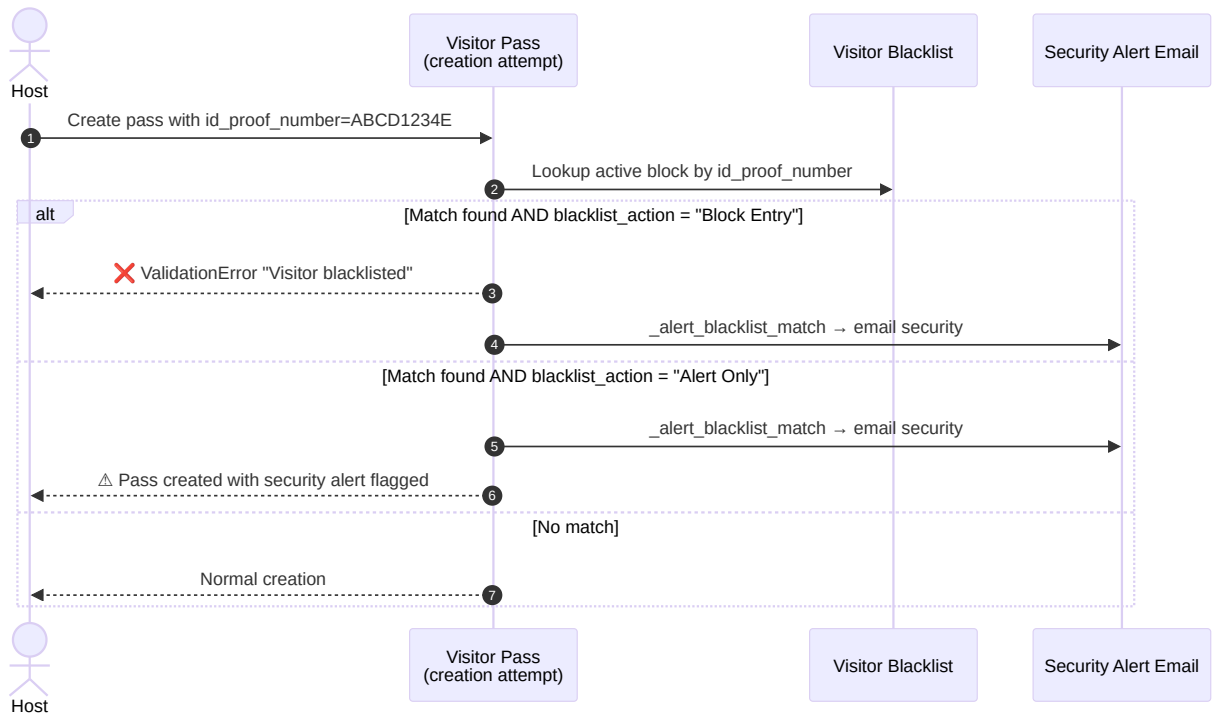
Mass-accountability flow during a fire alarm, gas leak, or evacuation drill.



The "Active Emergency Events" and "Missing in Muster" number cards on the Security Dashboard surface this in real time.

8. Visitor Blacklist Match

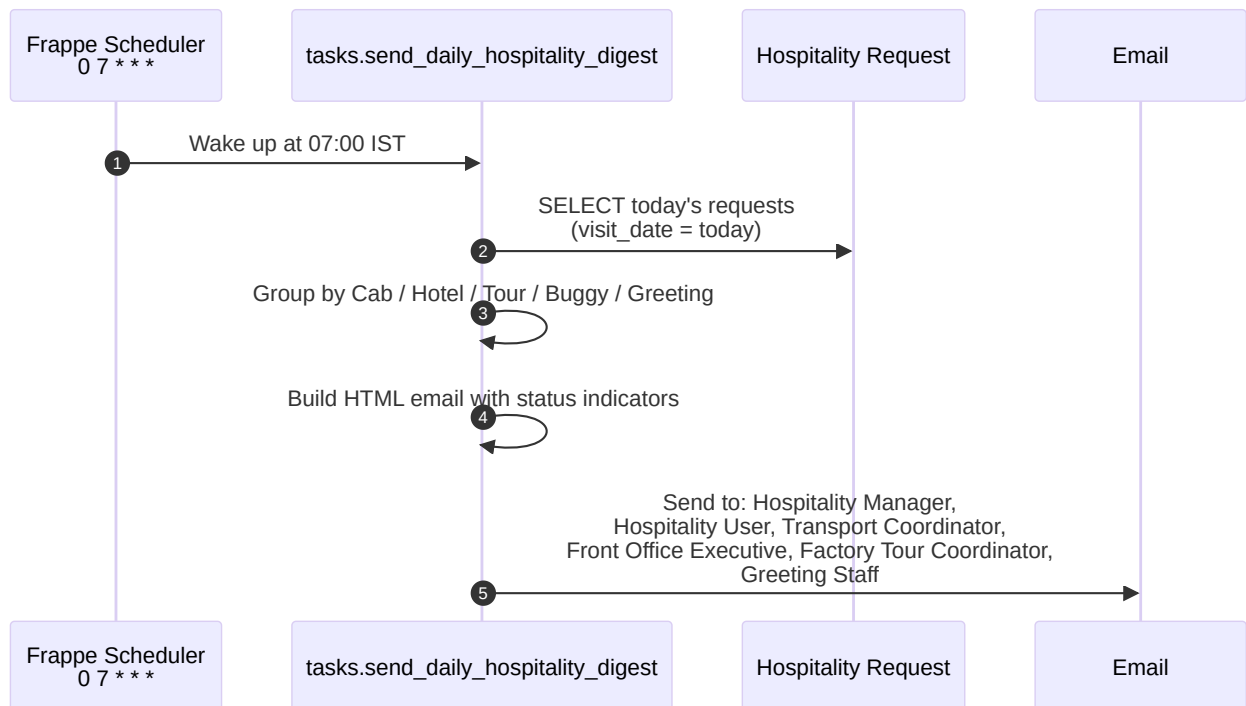
Defensive flow that fires automatically when someone tries to create a pass for a blacklisted person.



Both behaviour modes are tunable from VMS Settings → `blacklist_action`.

9. Daily Hospitality Digest

Automated digest that lands in hospitality team inboxes every morning.

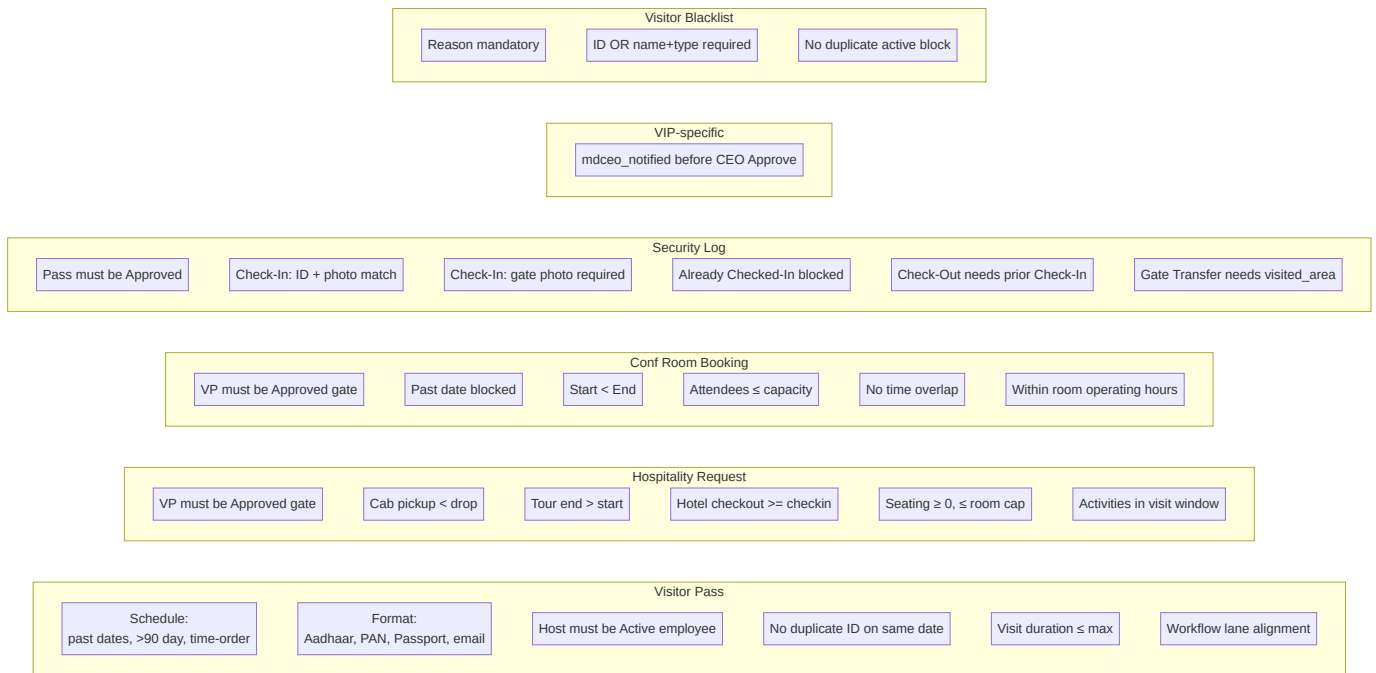


Each section in the email links back to the Hospitality Request so staff can click in and update status.

Validations

Every business rule the system enforces. Each rule is named, located in source, shown with the trigger that fires it, and quoted with the exact error message the user sees.

At a glance



Visitor Pass — 10 rules

Defined in `visitor_pass.py`.

#	Rule	Method	Trigger	Error message
1	Past visit_date rejected	_validate_schedule	validate()	Visit date {date} is in the past. Pick today or a future date.
2	>90 day future visit_date rejected	_validate_schedule	validate()	Visit date cannot be more than 90 days in the future.
3	expected_checkin must be < expected_checkout	_validate_schedule	validate()	Expected Check-In ({hh:mm:ss}) must be before Expected Check-Out ({hh:mm:ss}).
4	Email must be valid format	_validate_formats	validate()	Email ID '{value}' is not a valid email address.
5	Aadhaar exactly 12 digits	_validate_formats	validate()	Aadhaar must be exactly 12 digits. Got: {raw}
6	PAN format ^[A-Z]{5}[0-9]{4}[A-Z]\$	_validate_formats	validate()	PAN Card format is 5 letters + 4 digits + 1 letter (e.g., ABCDE1234F). Got: {raw}
7	Passport length 6-12 chars	_validate_formats	validate()	Passport number should be 6-12 characters. Got: {raw}
8	Host must exist + be Active	_validate_host_active	validate()	Host {emp} is not an Active employee (status: {status}). Cannot assign pass.
9	No duplicate ID on same date	_validate_duplicate_pass	validate()	A visitor pass ({existing_name}) already exists for this ID Proof on {date}. Duplicate passes are not allowed.
10	Visit duration ≤ max_visit_duration_hrs	_validate_visit_duration	validate()	Visit duration ({hrs:.1f} hrs) exceeds the maximum allowed ({max} hrs). Adjust the expected check-in/out times.

Plus (workflow-stage validations)

Rule	When	Error
Visitor photo required before submit	before_submit()	Visitor Photo is required before submitting the pass.
ID proof scan required before submit	before_submit()	ID Proof Scan is required before submitting the pass.
mdceo_notified=1 required before CEO approval (VIP only)	before_submit()	MD/CEO must be notified before submitting a VIP Visitor Pass. Please check the MD/CEO Notified field in the VIP Details section.

Visitor Invitation — 5 rules

Defined in `visitor_invitation.py`.

#	Rule	Method	Error
1	Past visit_date rejected	<code>_validate_visit_date</code>	<i>Visit date {date} is in the past. Cannot send an invitation for a past date.</i>
2	Visitor email must be valid	<code>_validate_email</code>	<i>Visitor Email '{value}' is not a valid email address.</i>
3	Check-in before check-out	<code>_validate_time_range</code>	<i>Expected Check-In must be before Expected Check-Out.</i>
4	Host must be Active	<code>_validate_host_active</code>	<i>Could not find Host Employee: {emp}</i>
5	Invitation expiry must be future	<code>validate</code>	<i>Invitation Expiry must be a future date and time.</i>

Plus (send-time validations)

Rule	When	Error
Save before send	<code>send_invitation()</code>	<i>Save the Visitor Invitation before sending the invitation mail.</i>
Visitor email required to send	<code>send_invitation()</code>	<i>Visitor Email is required before sending invitation.</i>
Expiry > send time	<code>send_invitation()</code>	<i>Invitation Expiry must be later than the send time.</i>

Hospitality Request — 8 rules

Defined in `hospitality_request.py`.

#	Rule	Method	Trigger condition	Error
1	VP must be Approved before Pending 	<code>_validate_visitor_pass_approved</code>	workflow_state past Draft + VP not Approved	<i>Cannot send Hospitality Request {name} for approval — the linked Visitor Pass {vp} is still {status}. Hospitality preparation can only begin once the visitor is confirmed (Visitor Pass must be Approved).</i>
2	Seating capacity ≥ 0	<code>_validate_seating_capacity</code>	always	<i>Seating Capacity cannot be negative.</i>
3	Seating $\leq$ room capacity	<code>_validate_seating_capacity</code>	conference_room set	<i>Seating ({n}) exceeds room {room} capacity ({cap}). Pick a larger room.</i>
4	Cab pickup datetime required	<code>_validate_cab_timing</code>	cab_required + type in (Pickup, Both)	<i>Pickup datetime required when cab type includes Pickup</i>
5	Cab drop datetime required	<code>_validate_cab_timing</code>	cab_required + type in (Drop, Both)	<i>Drop datetime required when cab type includes Drop</i>
6	Cab drop $\geq$ pickup	<code>_validate_cab_timing</code>	both set	<i>Drop datetime cannot be before pickup datetime</i>
7	Tour end > tour start	<code>_validate_tour_safety</code>	factory_tour_required	<i>Tour end time must be after start time</i>
8	Hotel check-out $\geq$ check-in	<code>_compute_hotel_nights</code>	hotel_required + both dates	<i>Hotel check-out ({date}) cannot be before check-in ({date}).</i>

Plus

Rule	Error
Hotel check-in within visit window (1-day grace)	<i>Hotel check-in ({date}) is before the visit date ({visit_date}).</i>
Hotel check-out within visit window (1-day grace)	<i>Hotel check-out ({date}) is after the visit ends ({end_date}).</i>
Tour / buggy / greeting times within visit window	<i>{Activity} ({time}) is outside the visit window ({start} → {end}).</i>
Buggy double-booking blocked	<i>Buggy {n} is already booked for {datetime}.</i>

Conference Room Booking — 7 rules

Defined in `conference_room_booking.py`.

#	Rule	Method	Error
1	VP must be Approved before Pending  (only if visitor_pass linked)	<code>_validate_visitor_pass_approved</code>	<i>Cannot send Conference Room Booking {name} for approval — the linked Visitor Pass {vp} is still {status}. Room preparation can only begin once the visitor is confirmed (Visitor Pass must be Approved).</i>
2	Cannot book past date	<code>validate_schedule</code>	<i>Cannot book a room for a past date.</i>
3	Both Start + End time required	<code>validate_schedule</code>	<i>Both Start Time and End Time are required.</i>
4	Start < End	<code>validate_schedule</code>	<i>Start Time must be before End Time.</i>
5	Attendees ≤ room capacity	<code>validate_capacity</code>	<i>Expected attendees ({n}) exceeds room capacity ({cap}) for {room}.</i>
6	No time overlap with another booking on same room	<code>validate_overlap</code>	<i>Time conflict with booking {name} ({title}: {start} - {end}). Please choose a different time slot.</i>
7	Within room operating hours	<code>validate_operating_hours</code>	<i>{room} is available from {time}. Your start time {time} is too early. / ...is available until {time}. Your end time {time} is too late.</i>

Plus

Rule	Error
Min booking duration	<i>Minimum booking duration for {room} is {min} minutes.</i>
Max booking duration	<i>Maximum booking duration for {room} is {max} hours.</i>

Security Log — 6 rules

Defined in `security_log.py`.

#	Rule	Trigger	Error
1	Pass must be Approved (or beyond)	every save	Visitor {name} must be approved before check-in. (Current Status: {status})
2	Check-In requires photo_at_gate	event_type=Check-In	Capture a live gate photo before saving the visitor check-in.
3	Check-In requires id_proof_match=1	event_type=Check-In	Confirm that the visitor matches the ID proof before saving the visitor check-in.
4	Check-In requires pass_photo_match=1	event_type=Check-In	Confirm that the visitor matches the pass creation photo before saving the visitor check-in.
5	Already-Checked-In blocked from another Check-In	new + Check-In + status=Checked-In	Visitor {name} is already Checked-In.
6	Check-Out needs prior Check-In	event_type=Check-Out + status≠Checked-In	Visitor {name} must be 'Checked-In' before they can 'Check-Out'. (Current Status: {status})

Plus

Rule	Error
Gate Transfer requires visited_area	Visited Area is required for gate transfer tracking.
Already-Checked-Out blocked from Check-In	Visitor {name} has already Checked-Out and the pass is now inactive.

Visitor Blacklist — 4 rules

Defined in `visitor_blacklist.py`.

#	Rule	Error
1	Reason is mandatory (audit/compliance)	Reason is required for every blacklist entry (audit/compliance requirement).
2	EITHER id_proof_number OR (visitor_name + id_proof_type) required	Provide either ID Proof Number or Visitor Name + ID Proof Type to identify the blacklisted person.
3	Auto-stamps blocked_by (current user) and blocked_on (today)	(silent)
4	No duplicate active blacklist on same id_proof_number	An active blacklist entry ({existing}) already exists for this ID Proof Number.

Conference Room — 1 rule

Operating-hour fields default to `08:00-20:00`. The `validate_operating_hours` on Booking enforces them. No standalone validation on the Conference Room itself beyond Frappe's mandatory-field check on `room_name`, `location`, `capacity`.

Auto-side-effects (not validations, but worth knowing)

Trigger	Side effect	Where
Visitor Pass approved	Generate badge_number + QR code	<code>visitor_pass.on_submit()</code>
Visitor Pass approved with hospitality flag	Auto-create Hospitality Request	<code>lifecycle.ensure_hospitality_request()</code>
Visitor Pass workflow transition	Append Visitor Event Log entry	<code>lifecycle.log_visitor_event()</code>
Security Log Check-In	Update Visitor Pass.status, sync_compliance_check, sync_contact_trace	<code>security_log.before_save</code>
Security Log Check-Out	Update Visitor Pass.status, refresh compliance score	<code>security_log.before_save</code>
Emergency Event status=Active	Generate Evacuation Muster row per checked-in visitor	<code>lifecycle.generate_emergency_muster_records()</code>
Visitor Pass with meal_required	Auto-derive assigned_meal_slots from check-in/out window vs MEAL_WINDOWS	<code>lifecycle.derive_hospitality_meal_plan()</code>
Visitor Pass with factory_tour_required	Auto-set buggy_required=1	<code>visitor_pass.js</code> change handler
Visitor Pass with visitor_type=VIP	Auto-set greeting_required=1	(configured via demo / portal flow)

Email send fail-soft

Every `frappe.sendmail` call wrapped in try/except — if the SMTP server is down or the Email Account is misconfigured, the workflow still completes. The error is logged to `tabError Log`. Wrapped sends:

File	Method
<code>visitor_pass.py</code>	<code>_send_approval_email</code> , <code>_notify_food_dept</code> , <code>_alert_blacklist_match</code>
<code>security_log.py</code>	<code>_send_host_checkin_email</code>
<code>hospitality_request.py</code>	<code>_send_hospitality_assignment_mail</code>

Visitor Invitation send is **NOT** wrapped — that's a user-initiated send action where failing loudly is correct.

Regression suite

All 38 validation rules above are exercised by `tests/test_regression.py`. Run anytime:

```
bench --site <site> run-tests --module visitormanagement.tests.test_regression
```

Result: Ran 45 tests in 13s – OK.

The tests are grouped:

Class	Validations covered
TestPermissionMatrix	Role × doctype perm matrix, Security read-only
TestVisitorPassValidation	All 10 VP validation rules
TestVisitorPassWorkflow	All 5 visitor types' approval chains, VIP mdceo_notified
TestHospitalityRequestFlow	HR creation + workflow + VP-approved gate
TestConferenceRoomBookingFlow	CRB workflow + gate + internal-only path
TestSearchHelpers	Existing-Visitor-Pass dropdown + phone search
TestLifecycleHelpers	_combine_visit_datetime shape robustness, _overlaps_time_window
TestAutoCreation	Meal-flagged VP → auto Hospitality Request
TestWorkspaceAndDoctypes	Smoke: workspace + 3 workflows + every doctype meta loads
TestRemovedFeatures	Health Screening stays removed (DocType, table, columns, lifecycle imports)

If anyone weakens any validation in this doc, the corresponding test fails loudly.